MYMANGALIST

คู่มือภาพรวมทั้งระบบ

จากปุ่มบนหน้าจอ → Dart → PHP API → MySQL / ไฟล์ → ผลลัพธ์บนจอ

อ่านจากภาพ ไม่ต้องไล่อ่านโค้ดก่อน

ครอบคลุม Login, การค้นหา, CRUD Manga และตอน, Reader,
คอมเมนต์, คะแนน, รีพอร์ต, Admin และสิทธิ์ข้อมูล



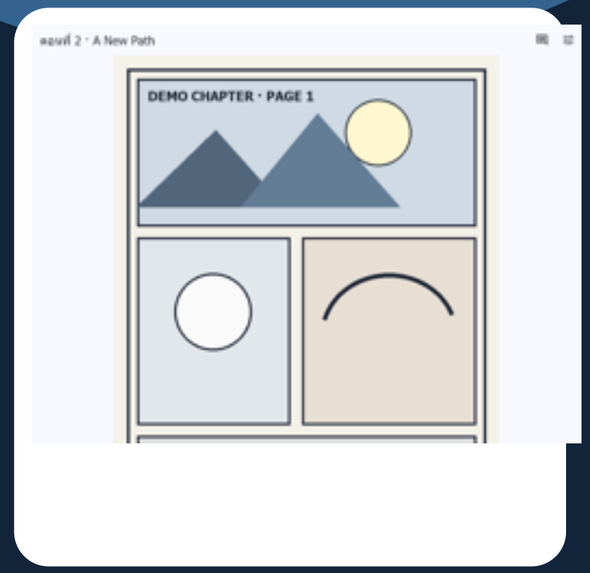
LOGIN



DISCOVER



DETAIL



READER

อ้างอิงโค้ดใน workspace ณ 14 ต.ค. 2569 • Flutter 2f95337 • API 1c8546c

4 ชั้นทีทำงานร่วมกัน

กดปุ่มหนึงครังไม่ได้คุยกับฐานข้อมูลตรง ๆ แต่ผ่าน PHP API ทีตรวจ token และสิทธิ์ทุกครัง

Flutter / Dart
Widget + State รับคลิก, ส่ง HTTP, วาดผล

→

PHP API
อ่าน JSON / multipart ตรวจ token, role, owner

→

MySQL
users, manga, chapters comments, reports ฯลฯ

→

File storage
ภาพปก, หน้า, PDF อยู่นอก htdocs

ตัวอย่าง Login

POST login.php ส่ง username/password → ตรวจ password hash → คืน access_token + user → DashboardPage

ตัวอย่างเปิดรายการ

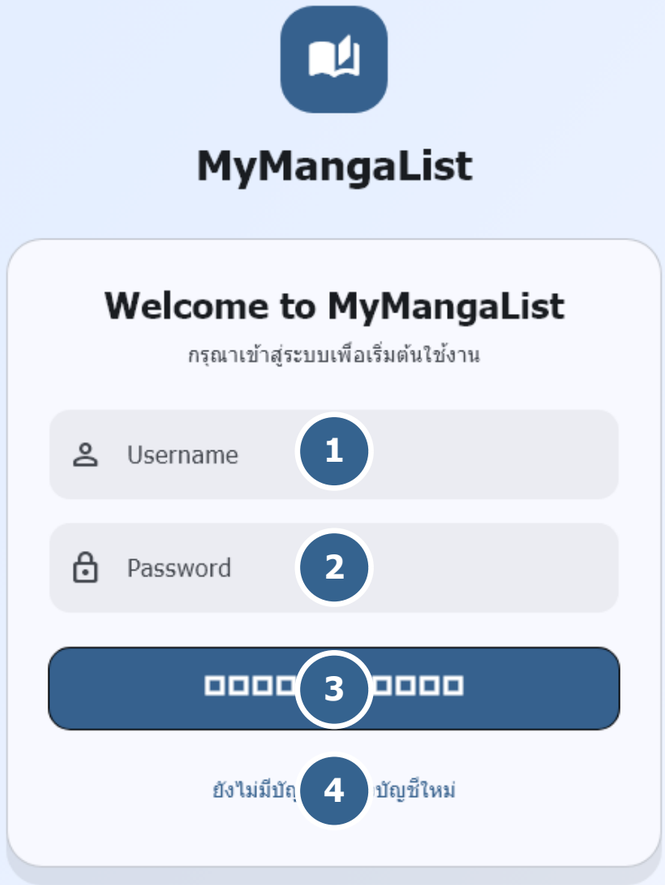
GET manga_get.php + Bearer token → require_auth.php → SELECT ตาม scope/user_id → JSON manga → MangaCard

ขอบเขตความเชือถือ (สำคัญทีสุด)

Flutter ซ่อนปุ่มตาม canEdit / role เพือการใช้งาน แต่ PHP ต้องอ่าน role จาก users และตรวจ user_id ของเจ้าของใน SQL อีกชัน ผู้ใช้ส่ง request เองได้ จึงห้ามเชือ role หรือ owner ที client แนบมา

03 เข้าสู่ระบบ

เริ่มแอปและรับ token สำหรับทุกหน้าที่ต้องล็อกอิน



1 Username

รับชื่อบัญชีและตัดช่องว่างก่อนส่ง

Dart: login.dart → _usernameController
API/ข้อมูล: POST login.php / users

2 Password

ตรวจว่ากรอกครบ แล้วส่งพร้อมชื่อบัญชีใน JSON

Dart: login.dart → _login()
API/ข้อมูล: password_verify กับ hash ใน users

3 เข้าสู่ระบบ

กดแล้วล็อกปุ่มระหว่างรอ; สำเร็จเปิด Dashboard

Dart: login.dart → http.post / Navigator
API/ข้อมูล: คืน access_token + user

4 สร้างบัญชีใหม่

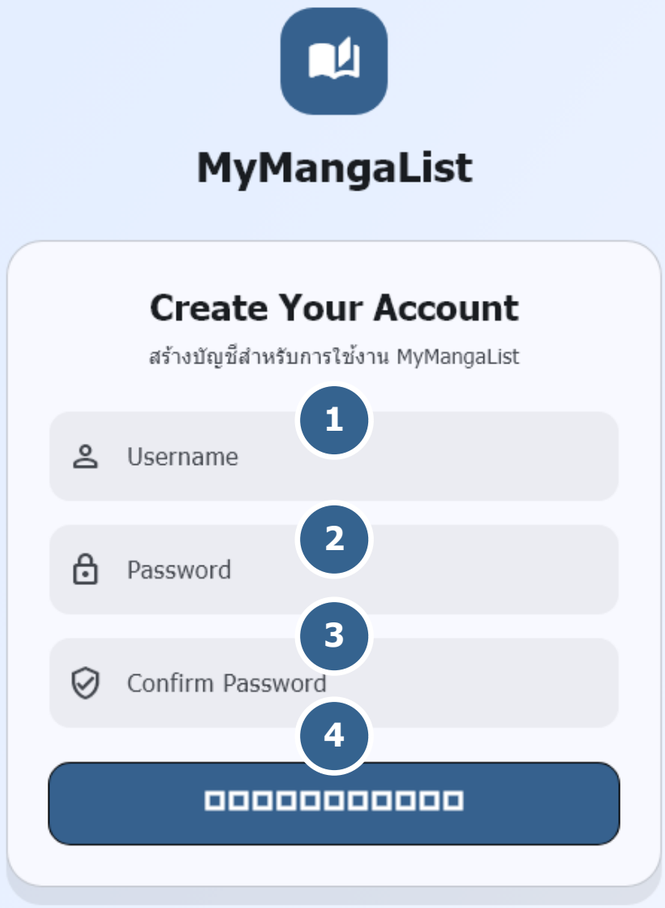
เปิด RegisterPage โดยยังไม่สร้างสิทธิ์ admin

Dart: login.dart → Navigator.push
API/ข้อมูล: register.php ใช้ role=user

เส้นทางข้อมูล

1. ส่ง JSON username / password
2. PHP สร้าง token 30 วัน เก็บเฉพาะ SHA-256 hash
3. Flutter เก็บ token ใน user map แล้วเปิด Dashboard

สร้างบัญชีใหม่



1 Username
ห้ามเว้นวรรค; ชื่อซ้ำถูกปฏิเสธ

Dart: register.dart → _register()
API/ข้อมูล: POST register.php / users

2 Password
ผู้ใช้ตั้งรหัสผ่าน; server เก็บเป็น hash

Dart: register.dart → jsonEncode
API/ข้อมูล: register.php → password_hash

3 Confirm Password
ต้องตรงกับ Password ก่อนส่ง

Dart: register.dart → validation
API/ข้อมูล: ส่ง confirm_password

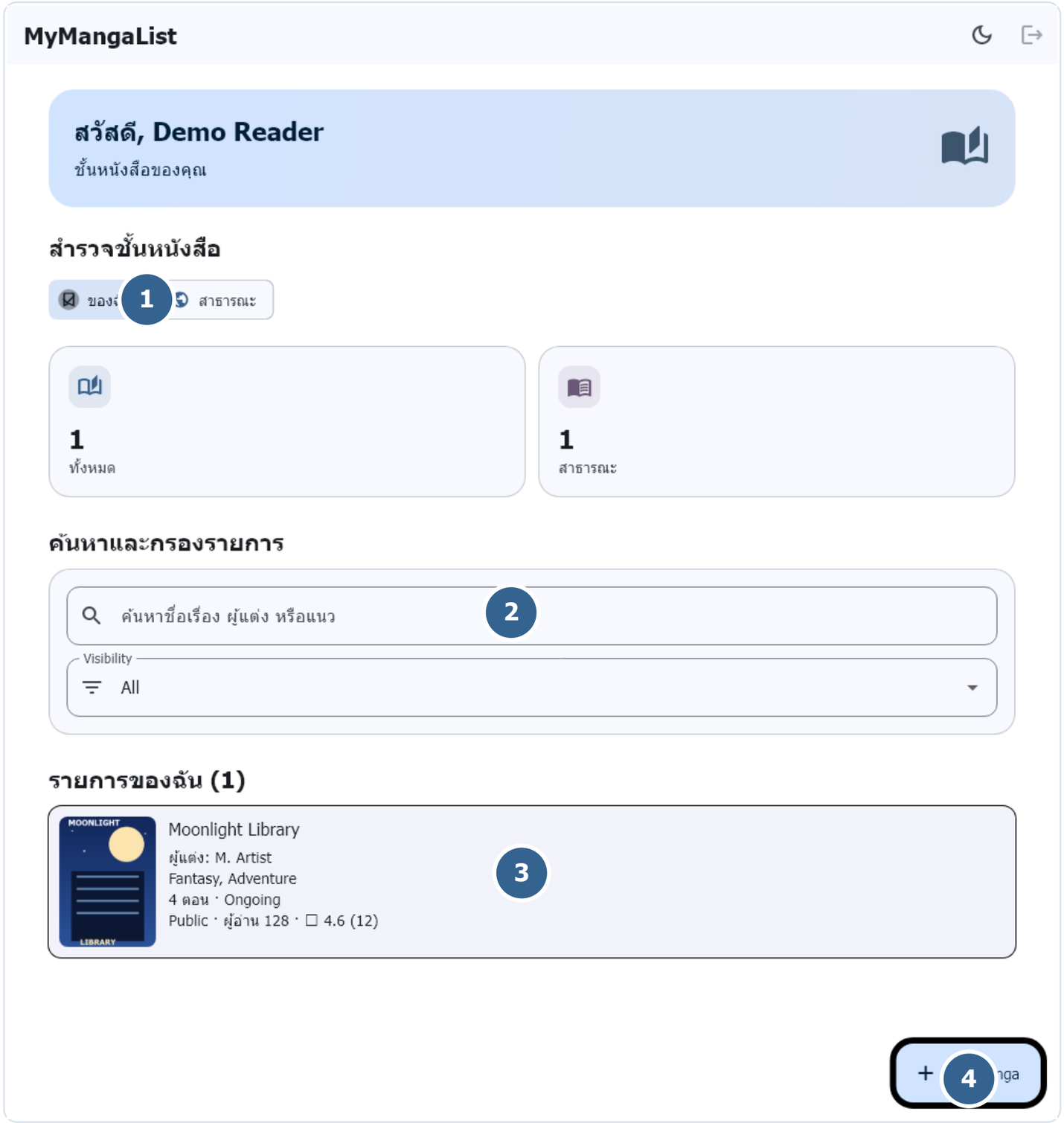
4 สร้างบัญชี
สำเร็จปิดหน้านี้ กลับไปให้ล็อกอิน

Dart: register.dart → Navigator.pop
API/ข้อมูล: INSERT users; role=user

เส้นทางข้อมูล
1. ส่ง JSON username / password / confirm_password
2. PHP ตรวจข้อมูล แล้ว INSERT users
3. รับ success/message → กลับหน้า Login

Dashboard · ของฉัน

ชั้นหนังสือของบัญชีตัวเองพร้อมค้นหาในหน้านี



1 ของฉัน / สาธารณะ

สลับ scope; ของฉันรวม Private และ Public ของเรา

Dart: dashboard.dart → _showPublic

API/ข้อมูล: GET manga_get.php หรือ ?scope=public

2 Search + Visibility

ค้น title/author/genre ในรายการที่โหลดแล้ว

Dart: dashboard.dart → _visibleManga

API/ข้อมูล: ใช้ JSON จาก manga_get.php

3 การ์ดมังงะ

ปก ผู้แต่ง แนว จำนวนตอน ยอดอ่าน คะแนน; แตะเปิด Detail

Dart: manga_card.dart / manga_cover.dart

API/ข้อมูล: manga_cover.php + manga_id

4 เพิ่ม Manga

เปิดฟอร์ม; บันทึก metadata แล้วอัปโหลดปกถ้ามี

Dart: dashboard.dart → _addManga()

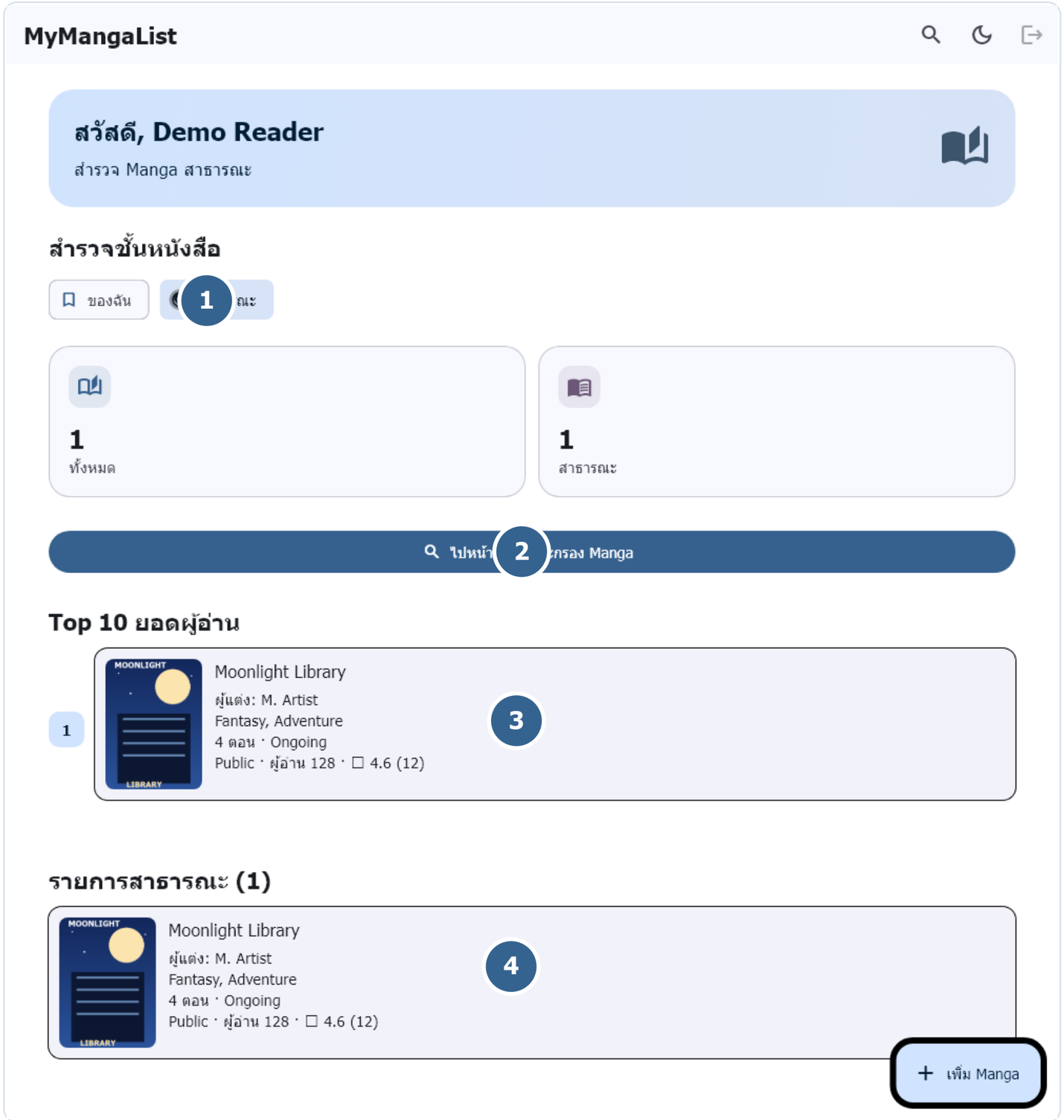
API/ข้อมูล: manga_add.php → cover_upload.php

เส้นทางข้อมูล

1. GET manga_get.php ใช้ Bearer token
2. PHP WHERE manga.user_id = user ที่ยืนยันแล้ว
3. JSON → setState → การ์ดของบัญชีนี้

Dashboard · สาธารณะ

สำรวจผลงานที่เปิดเผย พร้อม Top 10 จากยอดผู้อ่าน



1 สาธารณะ

เปลี่ยนเป็น scope=public; เรื่องซ่อนจะไม่อยู่ในผล

Dart: dashboard.dart → _showPublic
API/ข้อมูล: manga_get.php?scope=public

2 ไปหน้าค้นหา

ปุ่มนี้มีในมุมสาธารณะ ไม่ซ้ำในของฉัน

Dart: dashboard.dart → MangaSearchPage
API/ข้อมูล: หน้า Search โหลด public ใหม่

3 Top 10

เรียง view_count ก่อน แล้ว rating_average

Dart: dashboard.dart → _topManga
API/ข้อมูล: manga_views / manga_ratings

4 รายการสาธารณะ

แตะการ์ดเพื่อดูเรื่องของผู้อัปโหลดคนอื่น

Dart: dashboard.dart → _openManga()
API/ข้อมูล: manga_get.php คืน public ไม่ซ่อน

เส้นทางข้อมูล

1. GET public list พร้อม Bearer
2. SQL กรอง status=public AND is_hidden=0
3. Flutter sort และแสดง Top 10 + การ์ด

ค้นหาและกรอง

ค้นจากชื่อเรื่อง ผู้แต่ง แนว และสถานะการเผยแพร่



1 ช่องค้นหา

พิมพ์คำแล้วกรอง title/author/genres ใน client

Dart: manga_search.dart → _results
API/ข้อมูล: ข้อมูลเริ่มต้นจาก manga_get.php

2 Public / ของฉัน

เลือก scope ก่อนโหลดรายการใหม่จาก API

Dart: manga_search.dart → _public
API/ข้อมูล: GET ?scope=public หรือของฉัน

3 ประเภทการ์ตูน

เปิด checklist แนวทั้งหมด ไม่อิงแนวที่มีเรื่องใน DB

Dart: manga_search.dart / genre_options.dart
API/ข้อมูล: filter genres.any = OR

4 Release status

กรอง Coming Soon / Ongoing / Hiatus / Completed

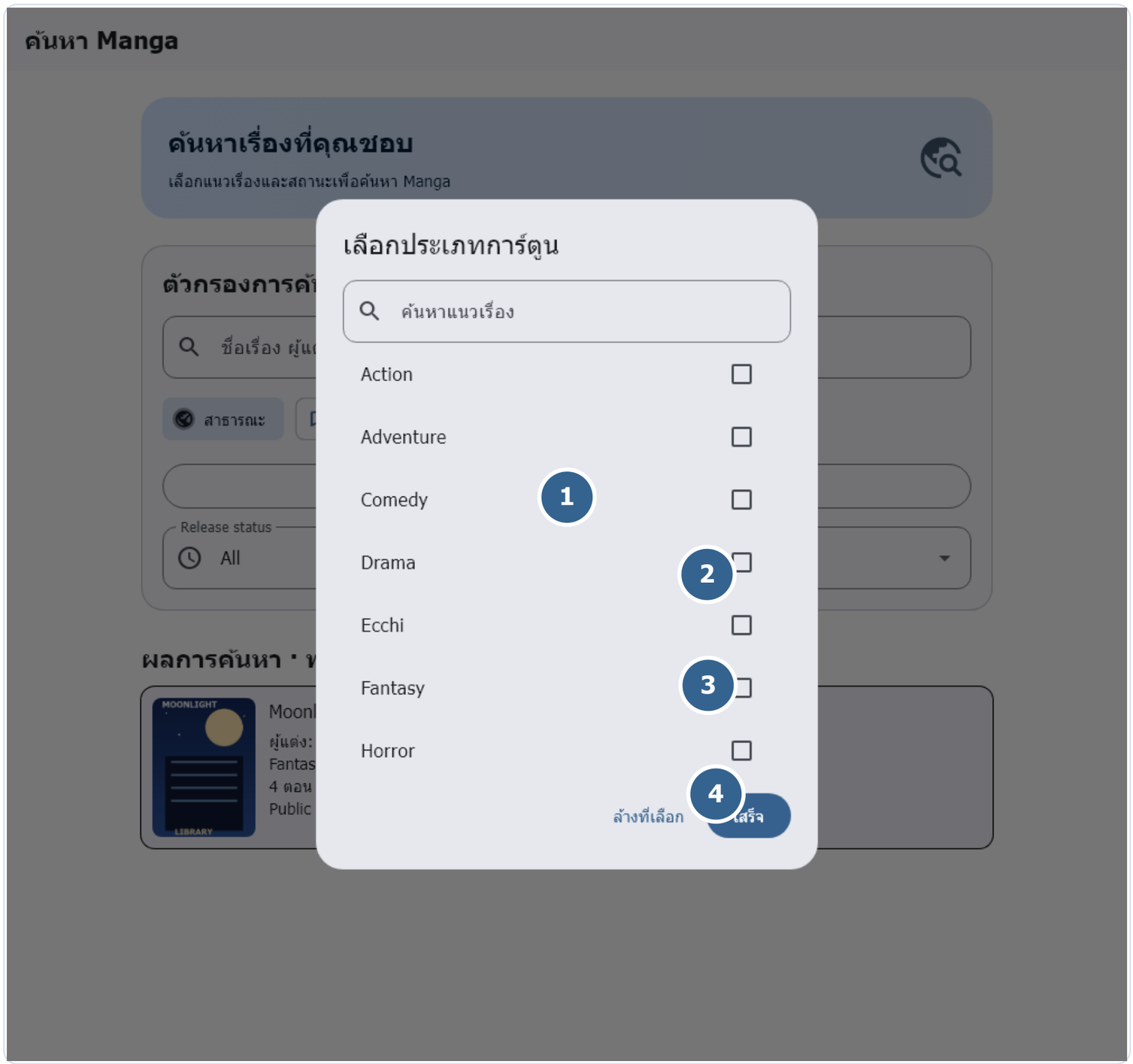
Dart: manga_search.dart → _release
API/ข้อมูล: ฟิลด์ manga.release_status

เส้นทางข้อมูล

1. GET รายการตาม scope ที่เลือก
2. Flutter ใช้ query + genre set + release filter
3. แตะผลค้นหา → MangaDetailPage

เลือกแนวเรื่อง

Checklist หลายแนว พร้อมช่องค้นหาภายในรายการ



1 ค้นหาแนว

ช่วยหาแนวในรายการหมวดหมู่ทีเตรียมไว้

Dart: manga_search.dart → _chooseGenres()
API/ข้อมูล: lib/genre_options.dart

2 ตีกหลายแนว

Set<String> เก็บค่าไม่ซ้ำ; ผลค้นหาตรงแนวใดแนวหนึง

Dart: manga_search.dart → _selectedGenres
API/ข้อมูล: client filter ด้วย genres.any

3 เลือนรายการ

มีหมวดให้เลือกแม้ยังไม่มีเรืองทีอัปโหลด

Dart: manga_search.dart → dialog ListView
API/ข้อมูล: ไม่ดึงตัวเลือกจาก manga_genres

4 เสร็จ

ปิด dialog แล้วแสดง chip แนวทีเลือก

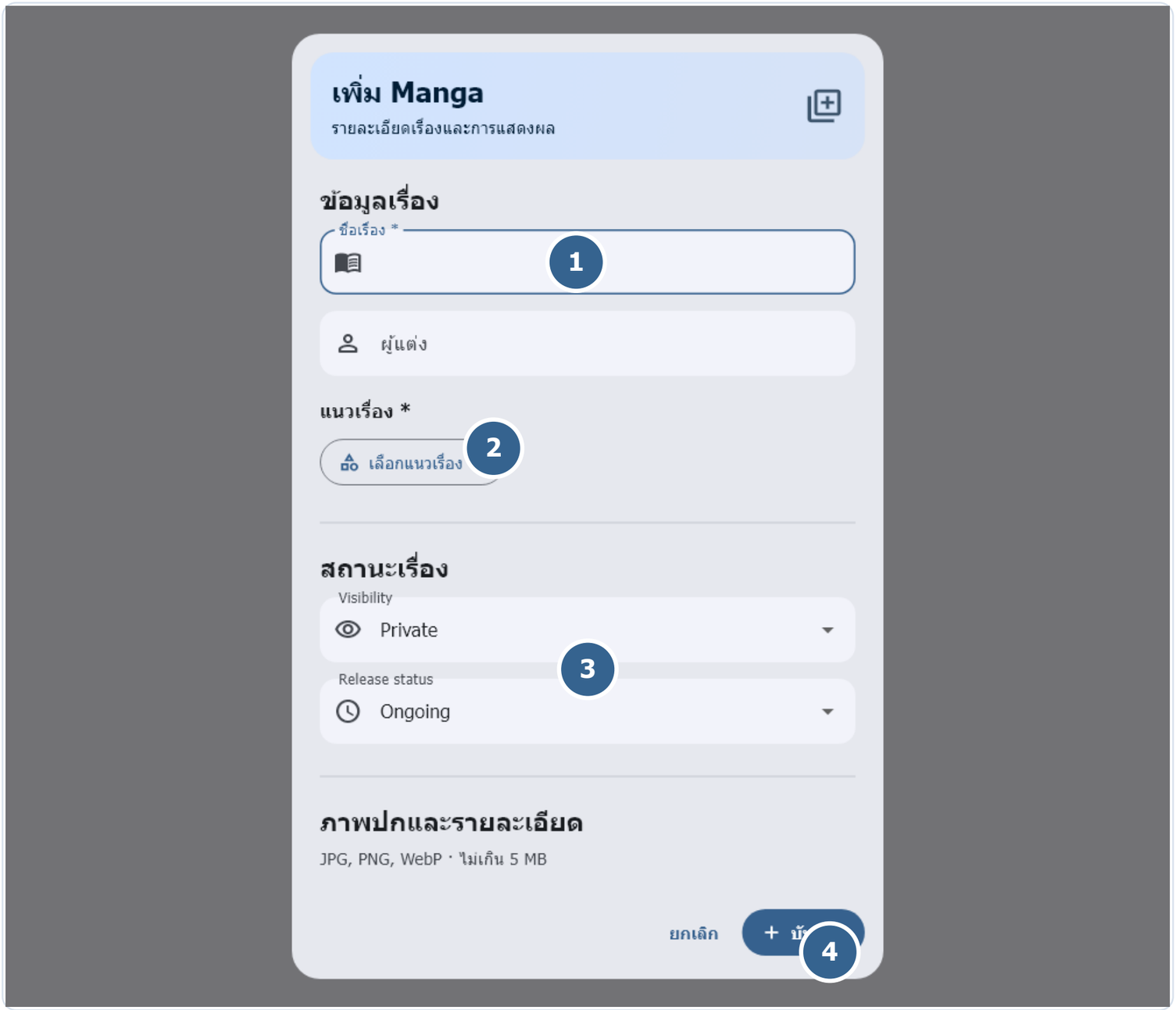
Dart: manga_search.dart → setState
API/ข้อมูล: ไม่ส่ง request ใหม่

เส้นทางข้อมูล

1. ตัวเลือกมาจาก genre_options.dart
2. ตีก/ค้นใน dialog ฝัง Flutter
3. _results กรองการ์ดจาก JSON เดิม

เพิ่ม Manga

สร้างเรื่องก่อน แล้วค่อยเพิ่มตอนทีละตอน



1 ชื่อเรื่อง / ผู้แต่ง

ชื่อเรื่องจำเป็น; ผู้แต่งช่วยค้นหาและแสดง Detail

Dart: add_manga_dialog.dart → fields
API/ข้อมูล: manga_add.php → manga

2 แนวหลายหมวด

เลือกมากกว่า 1 แนวจากรายการจริง

Dart: add_manga_dialog.dart → Set
API/ข้อมูล: manga_genres.php → manga_genres

3 Visibility / Release

Private/Public แยกจาก Ongoing/Coming Soon

Dart: add_manga_dialog.dart → _status/_releaseStatus
API/ข้อมูล: manga.status / release_status

4 บันทึก

ส่ง metadata; ปกเป็น multipart request แยก

Dart: dashboard.dart → _addManga()
API/ข้อมูล: manga_add.php → manga_cover_upload.php

เส้นทางข้อมูล

1. POST JSON title/author/genres/status/release
2. API INSERT manga + manga_genres คืน manga_id
3. ถ้ามีปก: POST multipart cover ≤5 MB

แก้ไข Manga

ฟอร์มเดียวกับเพิ่มเรือง แต่เติมข้อมูลเดิมและมีสถานะเพิ่ม



1 ข้อมูลเดิม

เปิด AddMangaDialog(manga: _manga) เพื่อเติมค่า

Dart: manga_detail.dart → _edit()
API/ข้อมูล: POST manga_update.php

2 แก้หลายแนว

บันทึกรายการ genres ใหม่ให้เรืองเดิม

Dart: add_manga_dialog.dart → fields
API/ข้อมูล: manga_update.php → manga_genres

3 Release status

ตอนแก้เลือก Hiatus / Completed เพิมได้

Dart: add_manga_dialog.dart → widget.manga
API/ข้อมูล: manga.release_status

4 บันทึก / ปกใหม่

แก้ metadata ก่อน; ถ้าเลือกปกใหม่ อัปโหลดอีกคำขอ

Dart: manga_detail.dart → uploadMangaCover
API/ข้อมูล: manga_cover_upload.php

เส้นทางข้อมูล

1. POST manga_update.php พร้อม manga_id
2. PHP WHERE manga_id AND user_id เจ้าของ
3. ถ้ามีปกใหม่ server แทนไฟล์เก่า

รายละเอียด Manga · เจ้าของ

เห็นปกซ้าย ข้อมูลขวา คำอธิบายและตอนด้านล่าง

รายละเอียด Manga



Moonlight Library

ผู้แต่ง: M. Artist

แนวเรื่อง: Fantasy, Adventure

Visibility: Public

Release status: Ongoing

จำนวนตอน: 3

ผู้อ่าน: 128

ความนิยม: ☐ 4.6 / 5 (12 คะแนน)

1

รายละเอียด

A journey through stories and magic.

2

ความคิดเห็นของเรื่อง

แก้ไข 3 ลบ

ตอน

4 ตอน

ตอนที่ 1 · The Door
เผยแพร่แล้ว

ตอนที่ 2 · A New Path
เผยแพร่แล้ว

ตอนที่ 3 · Draft
ฉบับร่าง

1 สรุปเรื่อง

ชื่อ ผู้แต่ง แนว Visibility Release ยอดอ่าน คะแนน

Dart: manga_detail.dart → _summaryContent
API/ข้อมูล: ข้อมูลจาก manga_get.php

2 รายละเอียด / คอมเมนต์

คำอธิบายอยู่แยกด้านล่าง; เปิดความคิดเห็นทั้งเรื่อง

Dart: manga_detail.dart → CommentsPage
API/ข้อมูล: comments_get.php?manga_id

3 แก้ไข / ลบ

เจ้าของเท่านั้นเห็นปุ่ม; ลบต้องยืนยัน

Dart: manga_detail.dart → _edit/_delete
API/ข้อมูล: manga_update/delete.php

4 เพิ่มและจัดการตอน

เพิ่มตอน เปิดอ่าน แก้/ลบตอนด้วยไอคอนท้ายแถว

Dart: manga_detail.dart → _saveChapter
API/ข้อมูล: chapters_get/add/update/delete.php

เส้นทางข้อมูล

1. เปิดการ์ดส่ง manga + access token
2. GET chapters_get.php ตรวจสิทธิ์เรื่อง
3. UI แสดงตอนที่อ่านได้และปุ่มเจ้าของ

รายละเอียด Manga · ผู้อ่าน

คะแนน 5 ดาว ความคิดเห็น รีพอร์ต และเปิดตอน

รายละเอียด Manga



Moonlight Library

ผู้แต่ง: M. Artist

แนวเรื่อง: Fantasy, Adventure

Visibility: Public

Release status: Ongoing

จำนวนตอน: 3

ผู้อ่าน: 128

ความนิยม: ☐ 4.6 / 5 (12 คะแนน)

1

รายละเอียด

A journey through stories and magic.

ให้คะแนนเรื่องนี้

★ ★ ★ ★ 2

ควา 3 องเรื่อง

4 langa

ตอน

ตอนที่ 1 · The Door
เผยแพร่แล้ว

ตอนที่ 2 · A New Path
เผยแพร่แล้ว

1 ข้อมูลผลงาน

ผู้อ่านเห็นชื่อ ปก แนว จำนวนตอน และสถานะ

Dart: manga_detail.dart → _summaryContent
API/ข้อมูล: manga_get.php / cover.php

2 ให้คะแนน 1–5

คะแนนส่วนตัวปรับได้; API คืนค่าเฉลี่ยและจำนวนใหม่

Dart: manga_detail.dart → _rate()
API/ข้อมูล: manga_rate.php → manga_ratings

3 ความคิดเห็น

เปิดคอมเมนต์ระดับเรื่อง ไม่ผูก chapter_id

Dart: manga_detail.dart → CommentsPage
API/ข้อมูล: comments_get.php

4 รีพอร์ต Manga

ผู้อ่านแจ้งเหตุผลให้ admin ตรวจ

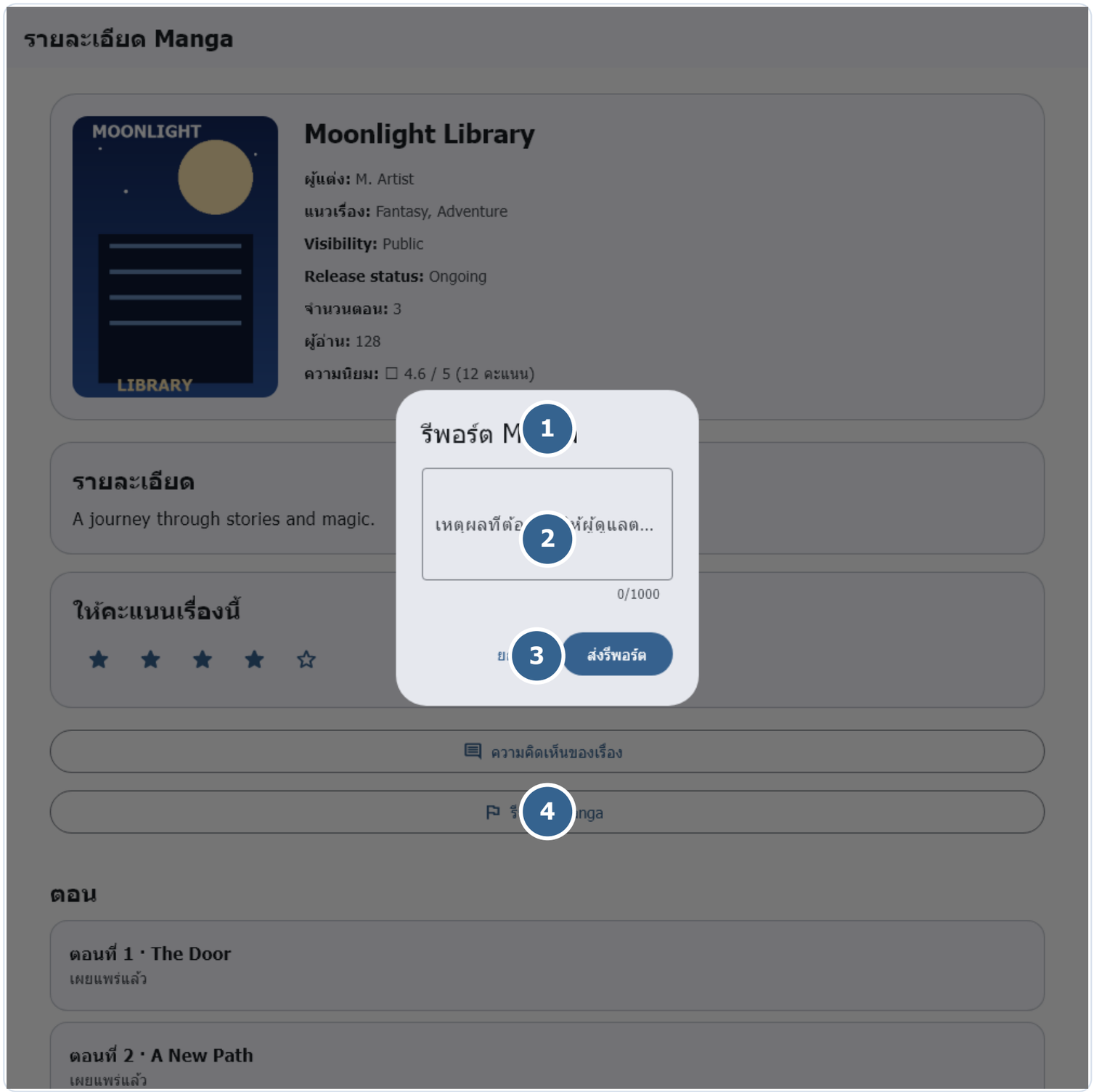
Dart: manga_detail.dart → _report()
API/ข้อมูล: report_manga.php → manga_reports

เส้นทางข้อมูล

1. ผู้อ่านเปิด public manga ที่ไม่ซ่อน
2. ให้คะแนน/รีพอร์ตต้องไม่ใช่เจ้าของ
3. API คืน JSON แล้วอัปเดต UI หรือคิว admin

รีพอร์ตผลงาน

ผู้ใช้ส่งเหตุผล แล้วผู้ดูแลเลือกว่าจะดำเนินการอย่างไร



1 เหตุผล

พิมพ์ 10–1000 อักษร; เรื่องเดียวกันมี pending ซ้ำไม่ได้

Dart: manga_detail.dart → _report()
API/ข้อมูล: report_manga.php

2 ส่งรีพอร์ต

ส่ง manga_id + reason พร้อม Bearer token

Dart: manga_detail.dart → _request()
API/ข้อมูล: INSERT manga_reports

3 สถานะ pending

ระบบเก็บผู้รายงาน ผู้ลงเรื่อง และชื่อเรื่องไว้

Dart: manga_detail.dart → SnackBar
API/ข้อมูล: manga_reports.status=pending

4 การตัดสินใจ

Admin ตรวจในแท็บรีพอร์ต ไม่ลบเรื่องทันที

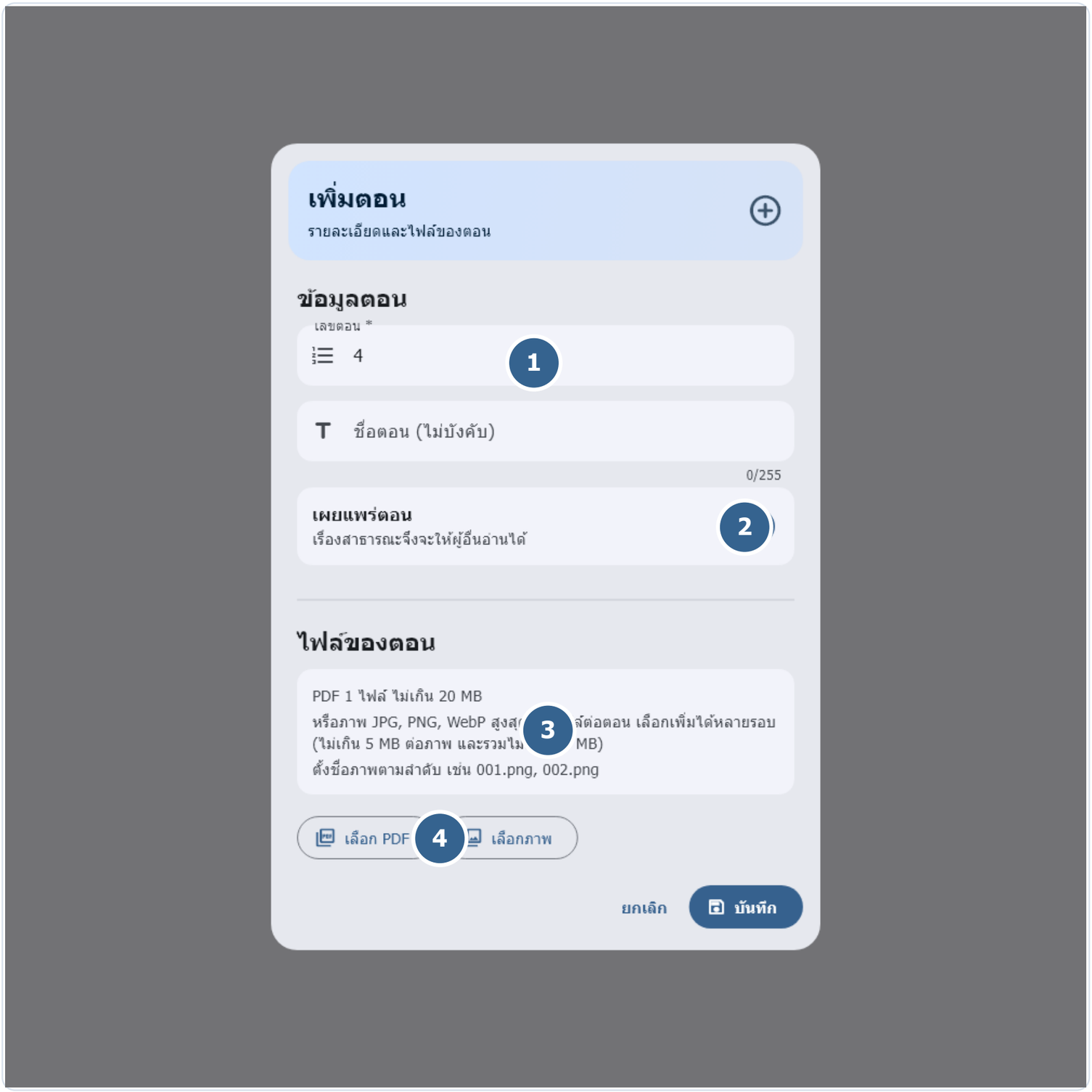
Dart: admin_dashboard.dart → _reportsView
API/ข้อมูล: admin_review_report.php

เส้นทางข้อมูล

1. POST {manga_id, reason}
2. PHP ตรวจ public / ไม่ใช่เจ้าของ / ไม่ซ้ำ pending
3. Admin Panel แสดงคิวรอตรวจ

เพิ่มตอนและอัปโหลด

เรืองหนึงมีหลายตอน แต่ละตอนเป็น PDF หรือชุดภาพ



1 เลขตอน / ชื่อตอน

เลขต้องเป็นบวกและไม่ซ้ำในเรือง

Dart: chapter_dialog.dart → _submit()
API/ข้อมูล: chapter_add.php → chapters

2 เผยแพร่ตอน

Draft ยังไม่เปิดให้ผู้อืนอ่าน; ตอนว่างเผยแพร่ไม่ได้

Dart: chapter_dialog.dart → published
API/ข้อมูล: chapter_update.php → is_published

3 ข้อจำกัดไฟล์

PDF 1 ไฟล์ ≤20 MB หรือภาพ ≤15 ไฟล์/ตอน

Dart: chapter_dialog.dart → _pickFiles()
API/ข้อมูล: manga_storage.php ตรวจ MIME/size

4 เลือก PDF / ภาพ

ภาพ JPG/PNG/WebP ≤5 MB ต่อไฟล์ รวม ≤30 MB

Dart: chapter_upload.dart → MultipartRequest
API/ข้อมูล: chapter_upload.php → chapter_files

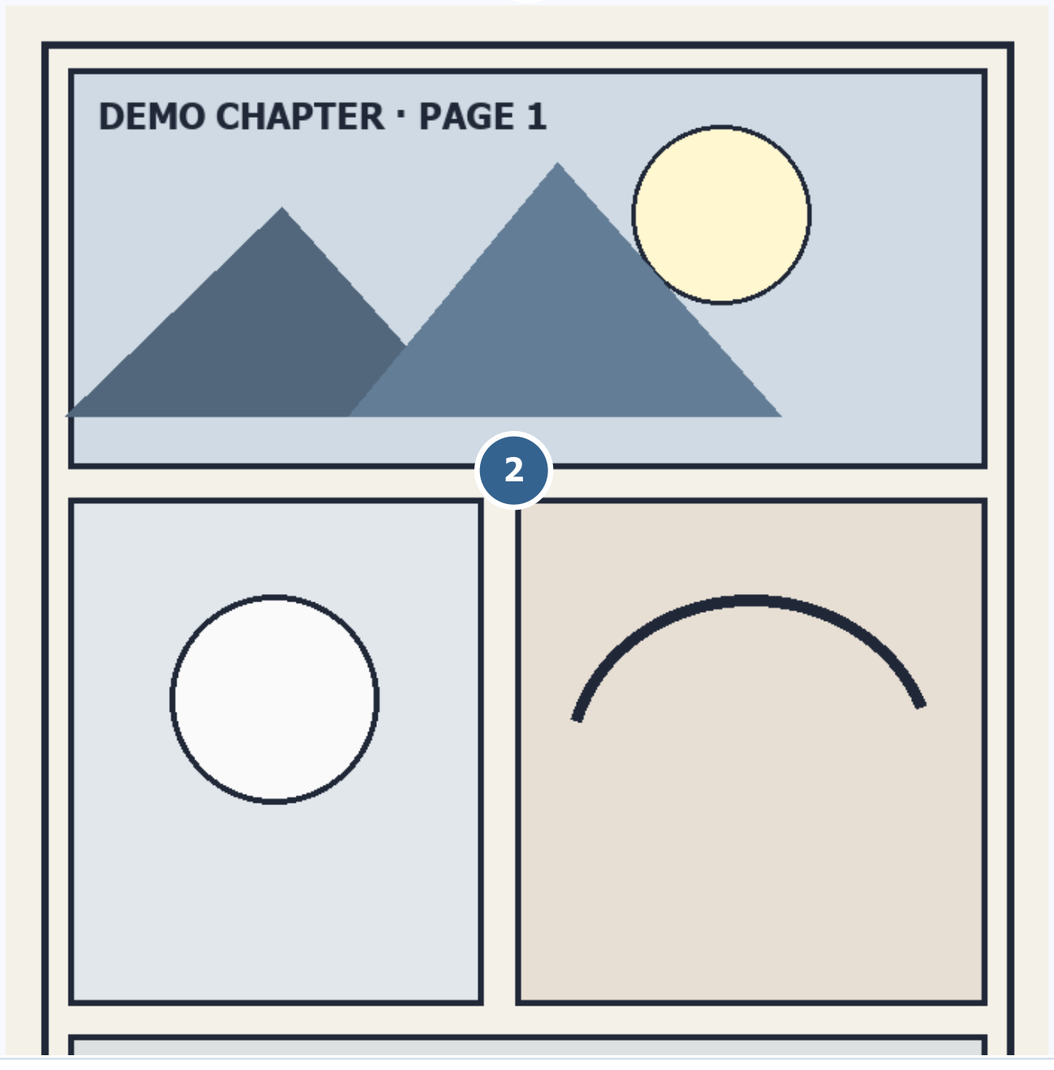
เส้นทางข้อมูล

1. POST chapter_add.php สร้าง draft คืน chapter_id
2. POST multipart chapter_upload.php บันทึกไฟล์/ลำดับ
3. POST chapter_update.php ตั้งชือ/เผยแพร่

หน้าอ่าน Manga

โหลด file_id ตามตอน แล้วขอภาพ/PDF ผ่าน API ที่ตรวจสิทธิ์

ตอนที่ 2 · A New Path



1 ชื่อตอน / ย้อนกลับ

รู้ chapter_id และรายการตอนที่อ่านได้จาก Detail

Dart: manga_reader.dart → MangaReaderPage
API/ข้อมูล: chapters_get.php

2 หน้าการ์ตูน

ภาพตัวอย่างแทนไฟล์จริง; ตัวอ่านรองรับภาพหรือ PDF

Dart: manga_reader.dart → _reader()
API/ข้อมูล: chapter_files_get.php → manga_file.php

3 ความคิดเห็นตอน

คอมเมนต์เฉพาะตอนส่ง chapter_id เพิ่ม

Dart: manga_reader.dart → CommentsPage
API/ข้อมูล: comments_get.php?chapter_id

4 ตั้งค่าการอ่าน

เลือกเลื่อนลง/ทีละหน้า ความกว้าง และ fit width

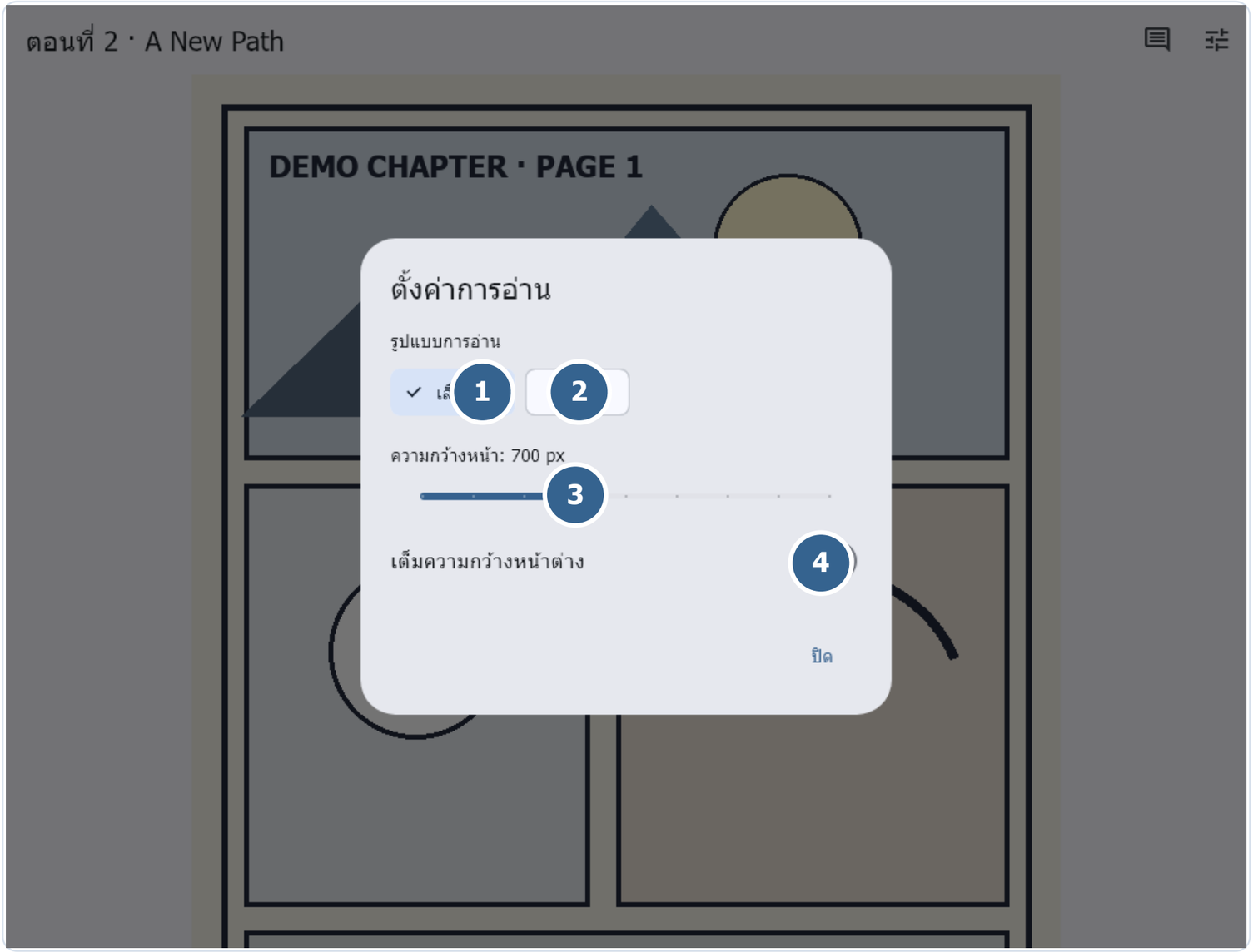
Dart: manga_reader.dart → _showSettings()
API/ข้อมูล: เปลี่ยน state ฝั่ง Flutter

เส้นทางข้อมูล

1. GET chapter_files_get.php → file_id ตาม sort_order
2. GET manga_file.php?file_id พร้อม Bearer
3. POST manga_view.php นับผู้อ่านไม่ซ้ำ

ตั้งค่าตัวอ่าน

ควบคุมขนาดหน้าและรูปแบบการอ่านโดยไม่เปลี่ยนไฟล์



1 เลื่อนลง

ภาพหลายหน้าต่อเนื่อง / PDF เลื่อนแนวดัง

Dart: manga_reader.dart → _singlePage=false
API/ข้อมูล: ไม่ส่ง API

2 ทีละหน้า

สลับ PageView / PDF แนวนอนและ snap

Dart: manga_reader.dart → _singlePage=true
API/ข้อมูล: ใช้ไฟล์ทีโหลดแล้ว

3 ความกว้าง

ปรับ 400–1200 px เพื่ออ่านสบายขึน

Dart: manga_reader.dart → _pageWidth
API/ข้อมูล: state เฉพาะ Reader

4 เต็มหน้าต่าง

ใช้ความกว้างพืนทีแสดงแทน slider

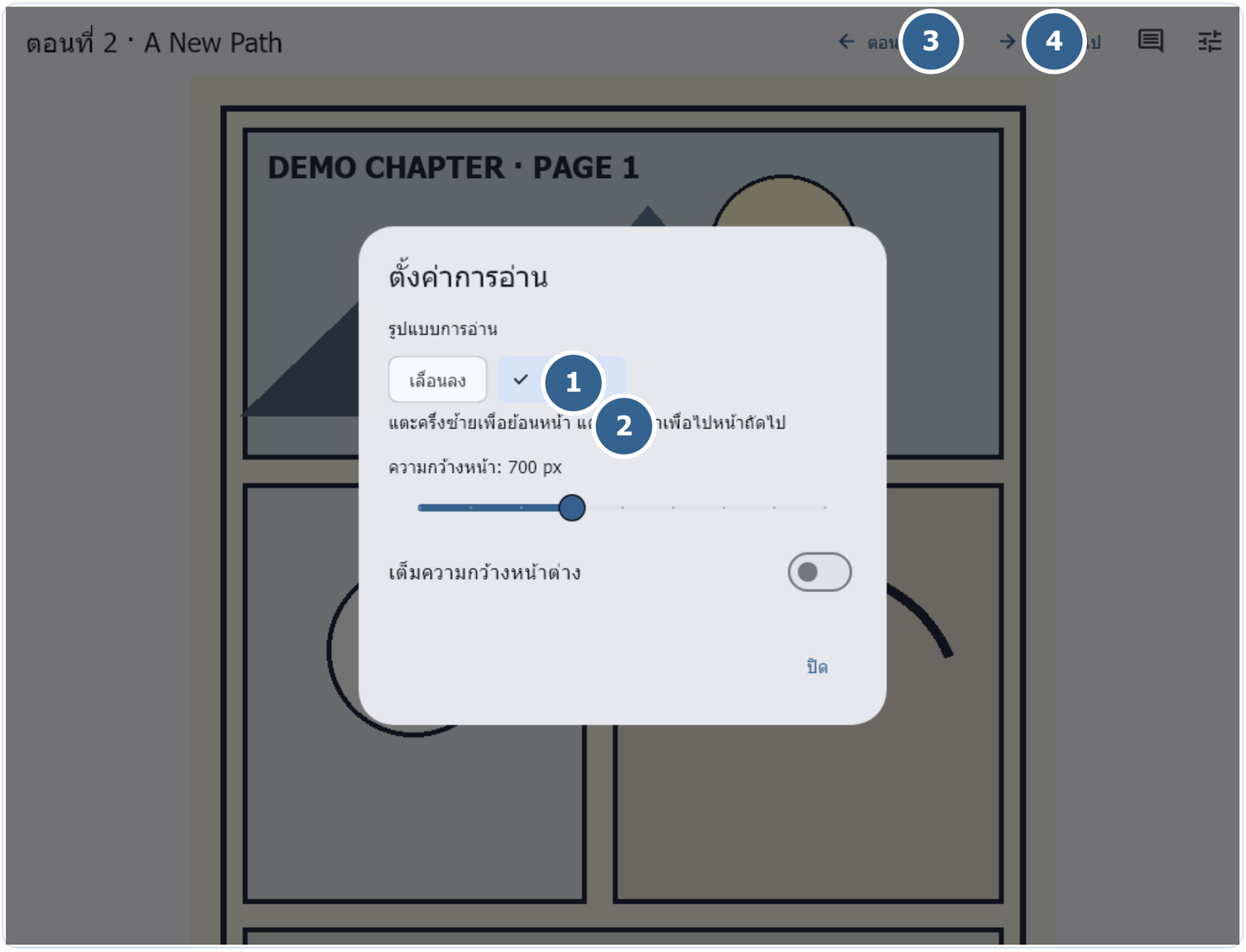
Dart: manga_reader.dart → _fitWidth
API/ข้อมูล: ไม่แก้ไฟล์ต้นฉบับ

เส้นทางข้อมูล

1. แตะปุ่มตังค่าบน AppBar
2. setState เปลียน layout ใน Reader
3. PDF ยัง render เป็น PNG คุณภาพสูง

โหมดแตะทีละหน้า

แตะซ้ายย้อนหลัง แตะขวาถัดไป และมีปุ่มเปลี่ยนตอน



1 เลือกทีละหน้า

เปิดการเปลี่ยนหน้าด้วยการแตะแบ่งครึ่งจอ

Dart: manga_reader.dart → _singlePage
API/ข้อมูล: ไม่มี request เพิ่ม

2 กติกาการแตะ

ตรวจการลาก/เวลาสัมผัสก่อนนับเป็น tap

Dart: manga_reader.dart → _tapToTurn()
API/ข้อมูล: เปลี่ยน _currentPage

3 ตอนก่อนหน้า

อยู่แถวเดียวกับตอนถัดไปเมื่อมีตอนก่อน

Dart: manga_reader.dart → _openPreviousChapter
API/ข้อมูล: Navigator.pushReplacement

4 ตอนถัดไป

แสดงเมื่ออ่านหน้าสุดท้ายและยังมีตอนต่อไป

Dart: manga_reader.dart → _openNextChapter
API/ข้อมูล: ใช้ chapters จาก Detail

เส้นทางข้อมูล

1. ผู้ใช้เลือกโหมดทีละหน้า
2. แตะซีกซ้าย/ขวาเปลี่ยนหน้าในตอน
3. เมื่อจบตอน กดเปลี่ยนตอนบน AppBar



1 รายการคอมเมนต์

แสดงผู้เขียน เนื้อหา และเวลาเรียงตามที่บันทึก

Dart: comments_page.dart → _load()
API/ข้อมูล: comments_get.php → comments

2 ลบความเห็น

เจ้าของความเห็น เจ้าของเรื่อง หรือ admin ลบได้

Dart: comments_page.dart → _post()
API/ข้อมูล: comment_delete.php ตรวจสิทธิ์

3 พิมพ์ความเห็น

ส่ง body ผูก manga_id; เพิ่ม chapter_id เมื่อตอบในตอน

Dart: comments_page.dart → _add()
API/ข้อมูล: comment_add.php

4 ส่ง

บันทึกสำเร็จแล้วโหลดรายการซ้ำ

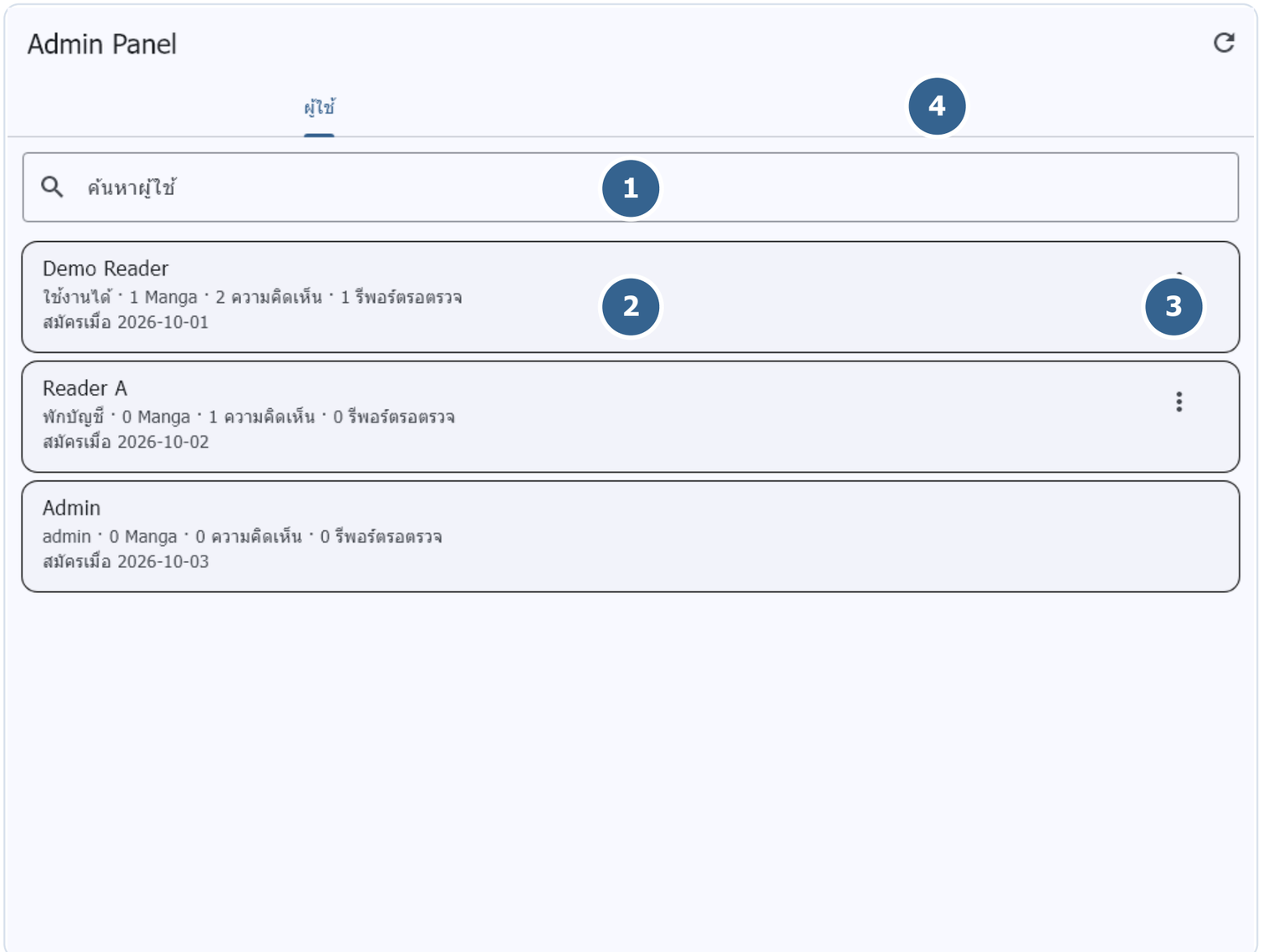
Dart: comments_page.dart → _load()
API/ข้อมูล: POST comment_add.php

เส้นทางข้อมูล

1. GET comments_get.php + manga_id[/chapter_id]
2. POST comment_add.php {manga_id,chapter_id,body}
3. PHP INSERT comments; UI refresh

19 Admin · จัดการผู้ใช้

ค้นหาบัญชี เห็นสถิติ และเปิดเมนูการจัดการ



1 ค้นหาผู้ใช้

กรอง username ในรายการที่โหลดแล้ว

Dart: admin_dashboard.dart → _search
API/ข้อมูล: GET admin_get_users.php

2 แถวผู้ใช้

แสดง role, จำนวน Manga/คอมเมนต์/รีพอร์ต

Dart: admin_dashboard.dart → _usersView()
API/ข้อมูล: users + COUNT จากตารางลูก

3 เมนูจัดการ

บัญชี user มีพัก/ปลดพักและลบ; admin ไม่มีเมนูนี้

Dart: admin_dashboard.dart → PopupMenuButton
API/ข้อมูล: require_admin.php

4 แท็บรีพอร์ต

สลับดูคิวรายงานโดยไม่ออก Admin Panel

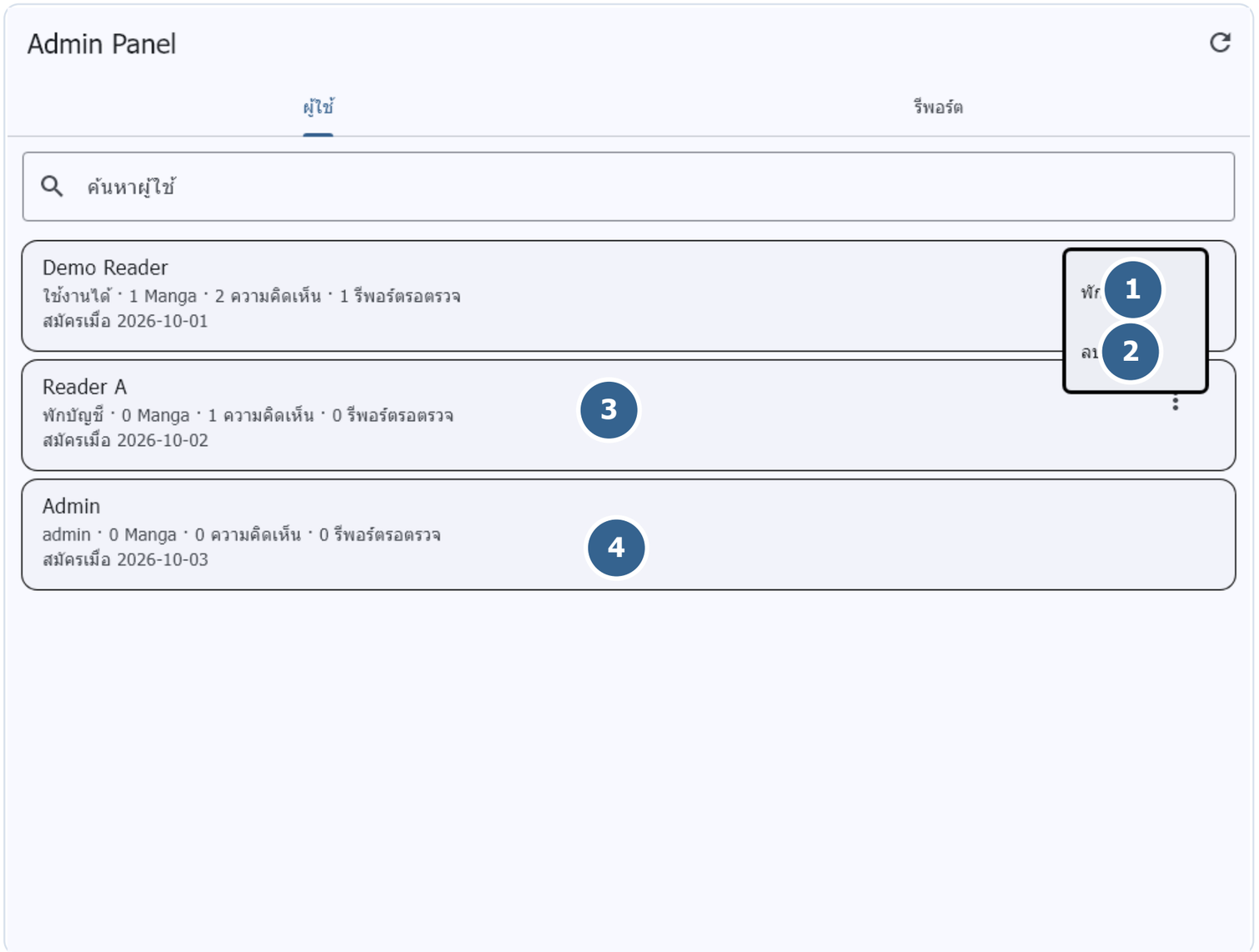
Dart: admin_dashboard.dart → TabBar
API/ข้อมูล: GET admin_get_reports.php

เส้นทางข้อมูล

1. GET admin_get_users.php + Bearer
2. require_admin อ่าน role จาก users
3. กด action → suspend/delete แล้ว refresh

Admin · เมนูผู้ใช้

การพักบัญชีและลบผู้ใช้เป็นคนละการกระทำ



1 พักบัญชี

สลับ is_suspended และยกเลิก token ของบัญชี

Dart: admin_dashboard.dart → _setSuspended()
API/ข้อมูล: admin_suspend_user.php

2 ลบผู้ใช้

มี dialog ยืนยัน ก่อนลบข้อมูลบัญชีและข้อมูลลูก

Dart: admin_dashboard.dart → _deleteUser()
API/ข้อมูล: admin_delete_user.php

3 บัญชีทีพัก

แสดงสถานะพัก และเมนูเปลี่ยนเป็นปลดพักบัญชี

Dart: admin_dashboard.dart → _usersView()
API/ข้อมูล: users.is_suspended

4 บัญชี admin

แสดงข้อมูลได้ แต่ไม่มีเมนูพัก/ลบใน UI

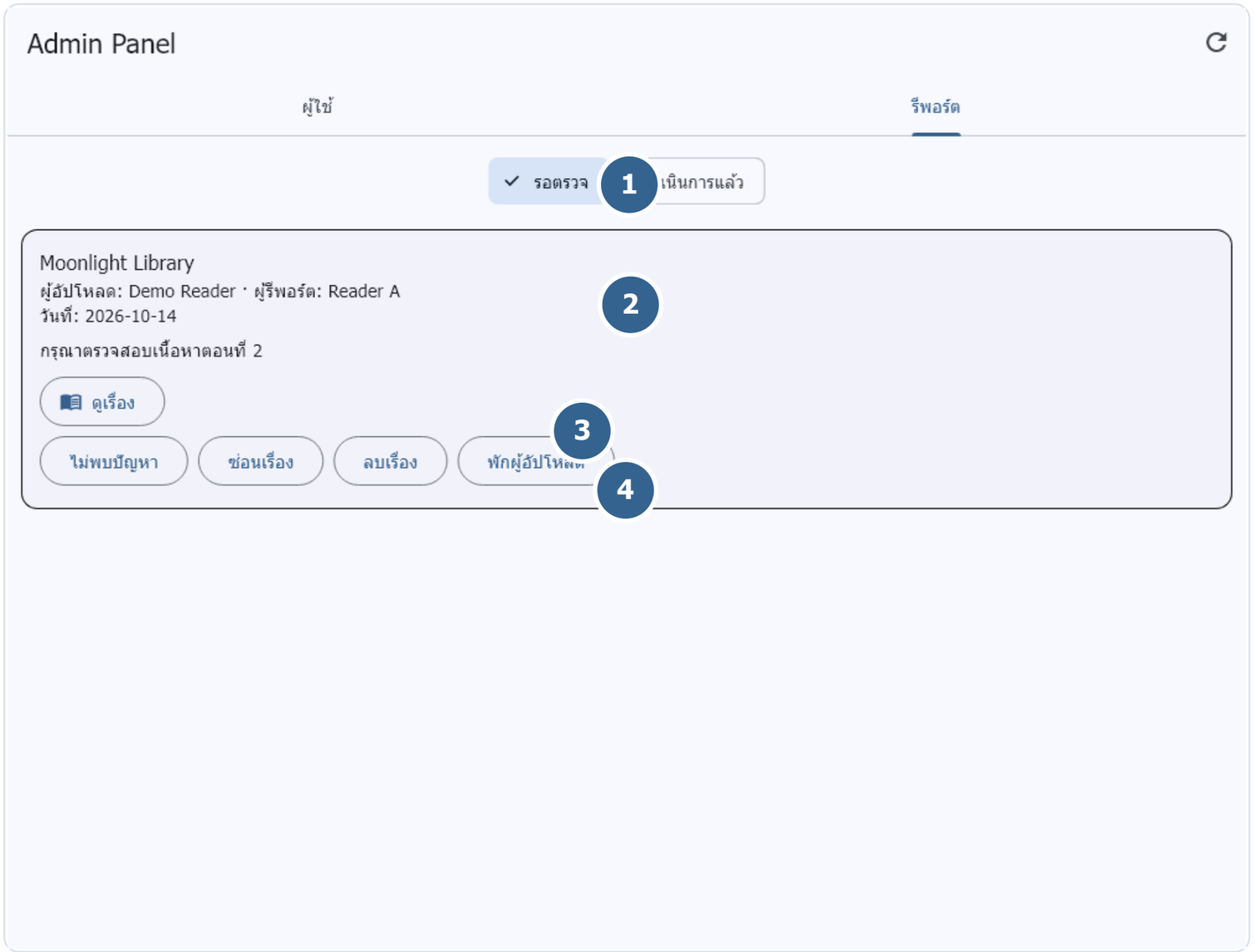
Dart: admin_dashboard.dart → isAdmin
API/ข้อมูล: require_admin.php ตรวจฝั่ง server

เส้นทางข้อมูล

1. POST admin_suspend_user.php {user_id,is_suspended}
2. หรือ POST admin_delete_user.php {user_id}
3. PHP ตรวจ admin แล้วอัปเดต/ลบข้อมูล

Admin · คิวรีพอร์ต

ผู้ดูแลพิจารณาเอง: ปิดเรือง ซ่อน ลบ หรือพักผู้ลง



1 รอตรวจ / ทำแล้ว

แยก pending กับ resolved ใน UI

Dart: admin_dashboard.dart → _showPending
API/ข้อมูล: GET admin_get_reports.php

2 ข้อมูลรีพอร์ต

เห็นเรือง ผู้ลง ผู้รายงาน เหตุผล และวันที

Dart: admin_dashboard.dart → _reportsView()
API/ข้อมูล: manga_reports + manga

3 ดูเรือง

เปิด Detail เพือตรวจเรืองทีถูกรายงาน

Dart: admin_dashboard.dart → _inspectReport()
API/ข้อมูล: manga_access.php ยอมให้ admin ตรวจ

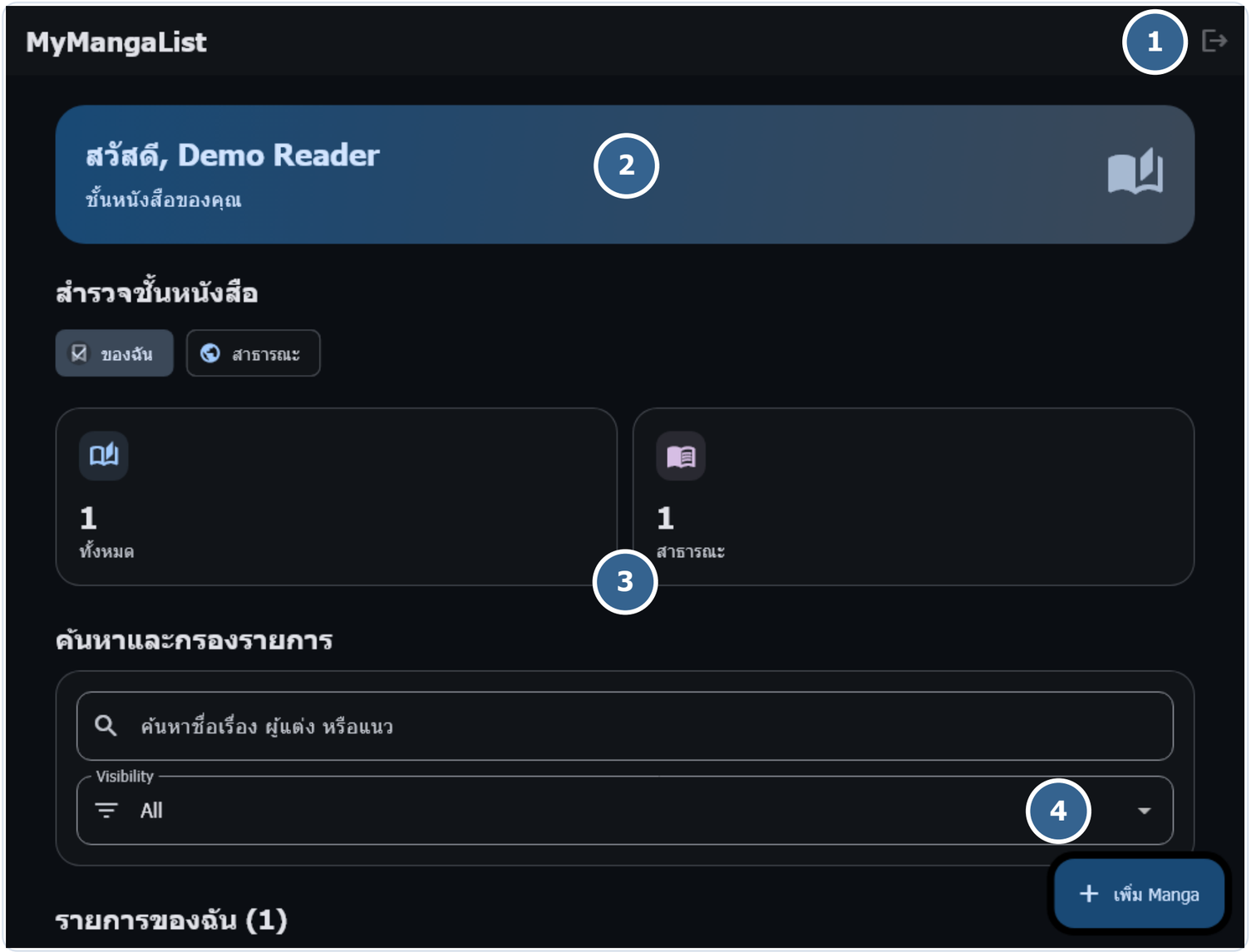
4 เลือก action

ไม่พบปัญหา / ซ่อน / ลบเรือง / พักผู้อัปโหลด

Dart: admin_dashboard.dart → _review()
API/ข้อมูล: admin_review_report.php

เส้นทางข้อมูล

1. POST {report_id, action} พร้อม admin token
2. require_admin.php + transaction ตัดสินใจ
3. manga_reports → resolved + action



1 ปุ่มดวงจันทร์

กดแล้วเปลี่ยน ThemeMode ของทั้งแอป

Dart: dashboard.dart → toggleAppTheme()
API/ข้อมูล: ไม่เรียก API

2 MaterialApp

ValueListenableBuilder สร้าง theme ใหม่เมื่อค่าเปลี่ยน

Dart: main.dart / theme_mode.dart
API/ข้อมูล: state ในหน่วยความจำ

3 การ์ดและฟอร์ม

UI ใช้ colorScheme จึงเปลี่ยนพื้น/ขอบ/ตัวอักษร

Dart: ThemeData.fromSeed
API/ข้อมูล: ข้อมูล Manga ไม่เปลี่ยน

4 ขอบเขต

ปัจจุบันไม่ได้บันทึกธีมลงฐานข้อมูลหรือเครื่อง

Dart: theme_mode.dart → ValueNotifier
API/ข้อมูล: เปิดแอปใหม่เริ่ม Light

เส้นทางข้อมูล

1. กด toggle → ValueNotifier เปลี่ยนค่า
2. MaterialApp รับ themeMode ใหม่
3. ทุกหน้าใช้ Theme สีทีเลือก

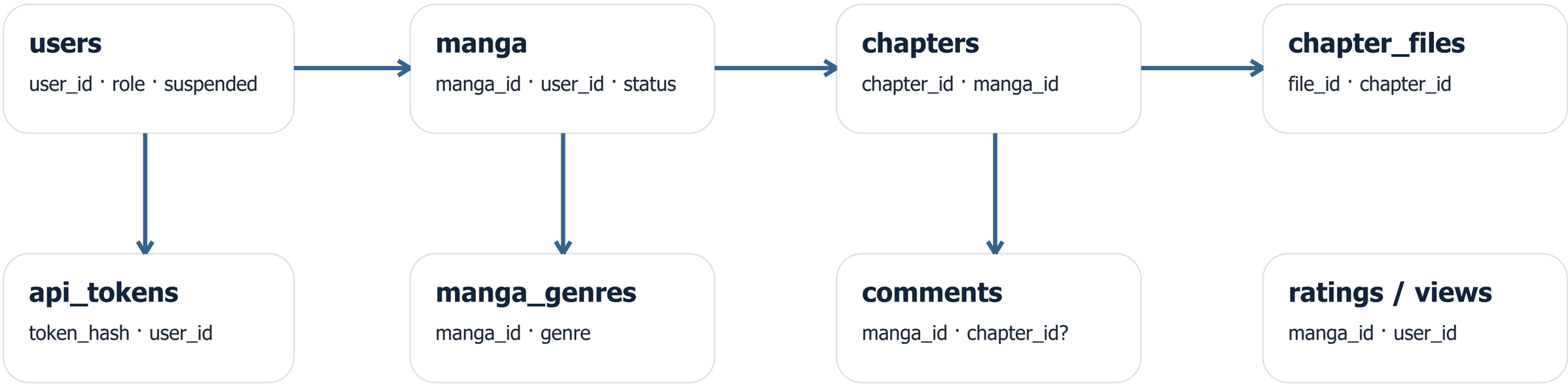
ฐานข้อมูล: เรือง → ตอน → ไฟล์

รหัสแต่ละระดับแยกกันเพื่อรองรับหลายเรือง หลายตอน หลายหน้า

ER map ย่อจาก database.sql

ความสัมพันธ์หลัก

ข้อมูลข้อความและ metadata อยู่ใน MySQL; ภาพ/PDF จริงอยู่บนดิสก์ฝั่ง server



สิ่งที่ต้องจำ

1 เรืองมีหลายตอน; 1 ตอนมีหลายไฟล์เรียงด้วย sort_order. manga_genres รองรับหลายแนว. comments ผูกทั้งเรืองหรือเฉพาะตอน. manga_reports บันทึกเหตุผลและผลการตัดสินใจ. ลบแถวแม่แล้ว foreign key cascade ลบข้อมูลลูกใน DB แต่ไฟล์จริงต้องให้ PHP ลบเอง

สิทธิ์: ใครทำอะไรได้

PHP เป็นผู้ตัดสินจาก token + role + user_id ไม่ใช่จากปุ่มบนหน้าจอ

ตารางสิทธิ์แบบใช้งานจริง

ผู้ใช้ทีไม่ล็อกอินเรียก API ป้องกันด้วย Bearer ไม่ได้; admin ดูเรืองทีถูกรายงานได้แม้ถูกซ่อน

การกระทำ	เจ้าของ	ผู้ใช้คนอืน	Admin	ไม่มี token
ดูเรือง Private	ได้	ไม่ได้	เฉพาะมีรีพอร์ต	ไม่ได้
ดูเรือง Public ไม่ซ่อน	ได้	ได้	ได้	ไม่ได้
เพิม/แก้/ลบเรืองและตอน	ได้	ไม่ได้	ไม่ใช่เจ้าของ: ไม่ได้	ไม่ได้
ให้คะแนน / รีพอร์ต	ให้คะแนนตัวเองไม่ได้	public ไม่ซ่อน	public และไม่ใช่เจ้าของ	ไม่ได้
จัดการผู้ใช้ / ตัดสินรีพอร์ต	ไม่ได้	ไม่ได้	ได้	ไม่ได้
อ่านไฟล์ตอน Draft	ได้	ไม่ได้	เฉพาะมีรีพอร์ต	ไม่ได้

Login ออก token อายุ 30 วันและเก็บเฉพาะ hash. require_auth.php ตรวจหมดอายุ/พักบัญชีทุก request; require_admin.php อ่าน role จาก users. Logout ใน Flutter กลับหน้า Login แต่ไม่มี endpoint revoke token แยกในเวอร์ชันนี

API map: ปุ่มไหนไปไฟล์ไหน

เปิด PDF คู่กับโค้ดแล้วไล่จากหน้าจอไป endpoint ได้ทันที

บัญชี

PHP: login.php · register.php

Data: users · api_tokens

Dart: login.dart · register.dart

ค้นหา/รายการ

PHP: manga_get.php · manga_cover.php

Data: manga · genres · ratings · views

Dart: dashboard.dart · manga_search.dart

เรือง

PHP: manga_add.php · manga_update.php · manga_delete.php · manga_cover_upload.php

Data: manga · manga_genres · storage

Dart: add_manga_dialog.dart · manga_detail.dart

ตอน/ไฟล์

PHP: chapter_add.php · chapter_update.php · chapter_delete.php · chapter_upload.php

Data: chapters · chapter_files · storage

Dart: chapter_dialog.dart · chapter_upload.dart

อ่าน

PHP: chapters_get.php · chapter_files_get.php · manga_file.php

Data: chapters · chapter_files

Dart: manga_reader.dart

สังคม

PHP: comments_get.php · comment_add.php · comment_delete.php

Data: comments

Dart: comments_page.dart

ความนิยม

PHP: manga_rate.php · manga_view.php

Data: manga_ratings · manga_views

Dart: manga_detail.dart · manga_reader.dart

Moderation

PHP: report_manga.php · admin_*.php

Data: manga_reports · users · manga

Dart: manga_detail.dart · admin_dashboard.dart

เส้นทางจริง 3 งานหลัก

ใช้เป็นสคริปต์อธิบายให้เพื่อนหรือผู้สอนเห็นภาพ

A · สร้างเรื่องและตอน

1. Dashboard + เพิม Manga → ฟอร์มคืน title, genres, status, release_status
2. POST manga_add.php → INSERT manga/manga_genres → คืน manga_id
3. ถ้ามีปก POST multipart cover; Detail + เพิมตอน → chapter_add.php คืน chapter_id
4. POST multipart chapter_upload.php → chapter_files; chapter_update.php → เผยแพร่

B · คนอืนค้นหาและอ่าน

1. Search GET manga_get.php?scope=public → JSON เฉพาะ public ไม่ซ่อน
2. แตะการ์ด → Detail GET chapters_get.php → เลือกตอน
3. Reader GET chapter_files_get.php → file_id; GET manga_file.php → bytes ภาพ/PDF
4. POST manga_view.php → นับ 1 ครังต่อ user และเรือง (ไม่รวมเจ้าของ)

C · รีพอร์ตจน Admin ตัดสิน

1. ผู้อ่าน POST report_manga.php {manga_id, reason} → pending
2. Admin GET admin_get_reports.php → ตรวจเรืองและเหตุผล
3. POST admin_review_report.php {report_id, action} → resolved
4. action: dismissed / hidden / manga_deleted / user_suspended

ภาพหน้าจอในเล่มนี้

เรนเดอร์จาก Flutter widget ปัจจุบันด้วย JSON และรูปสาธิต ไม่มีข้อมูลบัญชีจริง บางข้อความบนปุ่มหลักแสดงเป็นกล่องเพราะฟอนต์ใน test renderer; ชื่อปุ่มจริงระบุในบัตรคำอธิบาย

ซอร์สที่ตรวจ

Flutter: D:\myflutter69\lab6 (commit 2f95337) | PHP/SQL: C:\xampp\htdocs\mymangalist_api (commit 1c8546c). API URL ใน config.dart เป็น localhost สำหรับ Windows Desktop

ข้อจำกัดที่ควรพูดตรง ๆ

การเพิ่ม metadata กับอัปโหลดปก/ไฟล์เป็นคนละ request จึงมี partial success ได้. ยอดอ่านไม่เพิ่มทุกครั้ง: ใช้ INSERT IGNORE ต่อบัญชี และ error บันทึกยอดอ่านไม่ขัดการอ่าน. SQL cascade ลบแถว แต่ไฟล์กายภาพต้องลบด้วย PHP. ธีมมืดยังไม่ persist

PDF process ที่ใช้ประกอบ

MyMangaList_Deep_00_Overall.pdf; MyMangaList_Deep_01_Process_1_2.pdf; MyMangaList_Deep_02_Process_3_4.pdf; MyMangaList_Deep_03_Process_5_6.pdf. ใช้เป็นข้อมูลอ้างอิงและตรวจทานกับไฟล์ปัจจุบันก่อนเขียนเล่มนี

วิธีอ่าน: เริ่มที่หมายเลขบนภาพ → การกระทำในบัตรด้านขวา → ชื่อฟังก์ชัน Dart → endpoint PHP → ตาราง/ไฟล์ที่เปลี่ยน → กลับมาดูผลบนหน้าจอ

28 | เรียมแอป · Theme · API URL

จุดเริ่มร่วมของทุกหน้า ก่อน widget ใดจะเรียก PHP API

เปิดดูภาพหน้าจอเดิม: 02 และ 22

โครงสร้าง Flutter · ตัวแปรและ callback

lib/main.dart → main()

runApp(const MyApp()) คือจุดเริ่ม Flutter

MyApp.build() สร้าง MaterialApp

home: const LoginPage() เปิดหน้าเข้าสู่ระบบ

lib/main.dart → ValueListenableBuilder

ฟัง appThemeMode ซึ่งเป็น ValueNotifier<ThemeMode>

กำหนด theme, darkTheme และ themeMode

เมื่อค่าเปลี่ยน MaterialApp สร้างหน้าตาใหม่

lib/theme_mode.dart

เริ่มที่ ThemeMode.light

toggleAppTheme() สลับ dark ↔ light

ปุ่ม Dashboard เรียกฟังก์ชันนี้โดยตรง

lib/config.dart → ApiConfig

baseUrl = http://localhost/mymangalist_api

ทุกหน้าต่อชื่อ endpoint เข้ากับ baseUrl

localhost ในที่นี้ตั้งไว้สำหรับ Windows Desktop

ขอบเขตข้อมูล · API และผลลัพธ์

ภาพรวมโค้ดที่ใช้ซ้ำ

หน้าแต่ละหน้า import config.dart เมื่อเรียก API

หน้าที่ต้องยืนยันตัวตนส่ง Authorization header

Bearer token มาจากผลตอบกลับ login.php

ค่าธีมอยู่ตรงไหน

appThemeMode อยู่ในหน่วยความจำของแอป

สลับสีแล้ว widget อ่าน Theme.of(context)

รอบนี้ยังไม่มีการบันทึกธีมลงฐานข้อมูล

หน้าตา ↔ งานของระบบ

Widget แสดงช่องและปุ่ม; State เก็บค่าขณะใช้

callback อ่านค่า State แล้วส่ง HTTP request

setState() นำข้อมูลตอบกลับมาแสดงใหม่

เส้นแบ่งความรับผิดชอบ

Flutter ช่วยกรอกข้อมูลและแสดงผล

PHP ตรวจ token, role, owner และข้อมูลจริง

MySQL เก็บ metadata; ไฟล์ภาพ/PDF แยกต่างหาก

เส้นทางเมื่อใช้งานจริง

main() → MyApp → LoginPage → DashboardPage → หน้าอื่น ๆ; ทุก API ใช้ ApiConfig.baseUrl

29 | Login และ Register

สองหน้าฟอร์มทีรับข้อมูลบัญชี แล้วเปิดทางให้หน้าทีต้องมี token

เปิดดูภาพหน้าจอเดิม: 03–04

โครงสร้าง Flutter · ตัวแปรและ callback

lib/login.dart → LoginPage

_usernameController / _passwordController เก็บข้อความ

ปุ่ม Login เรียก _login()

_isLoading คุมสถานะกำลังส่งคำขอ

_login() → DashboardPage

ตรวจว่า 2 ช่องไม่ว่าง → http.post(login.php)

decode JSON และตรวจ statusCode กับ success

ใส่ access_token ใน user แล้ว pushReplacement

lib/register.dart → RegisterPage

controllers เก็บ username, password, confirm

_register() ตรวจฟอร์มแล้วส่ง register.php

ข้อความสำเร็จ/ผิดพลาดแสดงด้วย SnackBar

lib/auth_ui.dart

AuthLayout เป็นโครงหน้าฟอร์มร่วม

Login และ Register ส่ง title/children เข้าไป

ส่วนนีจัดหน้าตา ไม่ได้ตัดสินสิทธิบัญชี

ขอบเขตข้อมูล · API และผลลัพธ์

POST login.php

JSON request: username, password

JSON response: success, user, access_token

Flutter เก็บ token ไว้ใน user map ระหว่างใช้แอป

PHP login + api_tokens

ตรวจรหัสผ่านกับ users

ออก token 30 วันและเก็บ hash ใน api_tokens

ไม่เก็บ token ดิบในฐานข้อมูล

POST register.php

JSON request จากฟอร์ม: username, password

server สร้าง users.role = user เอง

ไม่มีช่องสมัคร admin และไม่รับ role จาก client

เมือ logout / เมือคำขอถัดไป

onLogout กลับ LoginPage และล้าง route ฝัง UI

หน้าถัดไปส่ง Bearer token ใน header

require_auth.php ตรวจ token และอ่าน role จาก DB ใหม่

เส้นทางเมือใช้งานจริง

TextField → controller → _login() → login.php → users/api_tokens → access_token → DashboardPage

30 | Dashboard · ของฉัน / สาธารณะ

หน้าเดียวสลับขอบเขตรายการ และแสดงการ์ดพร้อมจำนวนตอน ยอดชม และ Top 10

เปิดดูภาพหน้าจอเดิม: 05–06

โครงสร้าง Flutter · ตัวแปรและ callback

lib/dashboard.dart → DashboardPage

รับ user map จาก LoginPage

State: _manga, _topManga, _showPublic

initState() เรียก _loadManga()

_loadManga()

Future.wait() ขอรายการตามแท็บและสาธารณะ

แปลง data['manga'] เป็น List<Map>

เรียง Top 10 จาก view_count แล้ว rating_average

ตัวกรองและการ์ด

_visibleManga กรองสถานะและช่องค้นหาของฉัน

lib/manga_card.dart แสดงปก/ข้อมูลย่อ/จำนวนตอน

lib/manga_cover.dart โหลดภาพปกผ่าน API

ปุ่มนำทาง

_openManga() เปิด MangaDetailPage

ปุ่มค้นหาเปิด MangaSearchPage จากสาธารณะ

ปุ่มเพิ่มเรียก _addManga(); admin เปิด AdminDashboardPage

ขอบเขตข้อมูล · API และผลลัพธ์

GET manga_get.php

ไม่ใส่ scope = รายการของ user ที login

?scope=public = public และไม่ถูกซ่อน

response มี manga[] พร้อมค่ารวมของแต่ละเรือง

ข้อมูลบนการ์ดมาจากไหน

manga + users: title, author, cover, status

chapters: chapter_count ของตอนทีมองเห็นได้

manga_views / manga_ratings: ยอดชมและคะแนน

Top 10 คำนวณตรงไหน

PHP ส่ง view_count และ rating_average

Flutter เรียงรายการสาธารณะแล้ว take(10)

การเปลียนยอดชมจะเห็นหลังโหลดรายการใหม่

ขอบเขตความเป็นส่วนตัว

server ใช้ authenticatedUser.user_id คัดของฉัน

Flutter สลับแท็บเพือขอ scope ใหม่

ตัวกรองใน UI ไม่ใช่ตัวป้องกันข้อมูล

เส้นทางเมือใช้งานจริง

แท็บ/ค้นหา → _loadManga หรือ _visibleManga → manga_get.php → _manga → MangaCard → Detail

31 | ค้นหา · Checklist แนวเรือง

โหลดรายการตามขอบเขตก่อน แล้วกรองบนหน้าจอจากตัวเลือกทีผู้ใช้ตีก

เปิดดูภาพหน้าจอเดิม: 07–08

โครงสร้าง Flutter · ตัวแปรและ callback

lib/manga_search.dart → MangaSearchPage

State: _query, _manga, _public

_selectedGenres เป็น Set<String>

_release และ _visibility เก็บตัวกรองสถานะ

_load() → _results

_load() ขอ manga_get.php ตาม _public

_results กรองชือเรือง ผู้แต่ง และแนว

แนวหลายตัวใช้ genres.any(...) แบบตรงอย่างน้อย 1

_chooseGenres()

อ่าน mangaGenres จาก lib/genre_options.dart

TextField ค้นหาในรายการแนวทังหมด

CheckboxListTile เพิม/ลบค่าใน _selectedGenres

_open(manga)

เปิด MangaDetailPage พร้อม accessToken

canEdit เทียบ manga.user_id กับ user.user_id

กลับมาจากรายละเอียดแล้ว _load() ใหม่

ขอบเขตข้อมูล · API และผลลัพธ์

ข้อมูลดึงต้น

GET manga_get.php หรือ ?scope=public

response: manga[] และ genres[] ของแต่ละเรือง

รายการ checklist มาจากค่าคงทีในแอป

การกรองใน Flutter

ข้อความค้นหาแปลงเป็น lower-case

release_status เทียบ _release

ของฉันกรอง status private/public ได้

ความหมายของหลายแนว

ผู้ใช้ตีกได้หลายค่า

ผลลัพธ์ผ่านเมือเรืองตรงแนวใดแนวหนึง

แนวทียังไม่มี Manga ก็ยังแสดงใน checklist

สิทธียังอยู่ฝัง server

Flutter กรองเฉพาะรายการที API ส่งมาแล้ว

ไม่สามารถค้นพบ Manga private ของผู้อืน

endpoint ตรวจ token ก่อนส่งข้อมูล

เส้นทางเมือใช้งานจริง

ค้นหา/เลือกแนว → State เปลียน → _results → การ์ดผลลัพธ์ → _open() → MangaDetailPage

32 | เพิ่มและแก้ไข Manga

ฟอร์มเดียวใช้สองโหมด; metadata, แนวเรือง และภาพปกเดินคนละคำขอ

เปิดดูภาพหน้าจอเดิม: 09–10

โครงสร้าง Flutter · ตัวแปรและ callback

lib/add_manga_dialog.dart

AddMangaDialog(manga?) เติมค่าเดิมเมื่อแก้ไข

State เก็บ title, author, description, genres

_status = private/public; _releaseStatus เก็บสถานะ

_chooseGenre / _chooseCover / _submit

Checklist เลือกได้หลายแนวจาก genre_options.dart

ปกเลือก JPG/PNG/WebP และตรวจไม่เกิน 5 MB

_submit() คืน MangaFormResult(fields, coverFile)

สร้าง → lib/dashboard.dart

_addManga() รอผล showDialog

POST manga_add.php ด้วย JSON fields

ถ้ามีปก จึงเรียก uploadMangaCover(manga_id)

แก้ไข/ลบ → lib/manga_detail.dart

_edit() เปิด dialog พร้อม _manga เดิม

POST manga_update.php แล้วอัปโหลดปกถ้ามี

_delete() ยืนยันก่อน POST manga_delete.php

ขอบเขตข้อมูล · API และผลลัพธ์

metadata ไปฐานข้อมูล

fields: title, author, description, genres[]

status คือ private/public; release_status อีกช่อง

manga_add.php ใส่ user_id จาก token เท่านั้น

โครงสร้างหลายแนว

manga เก็บข้อมูลหลักของเรื่อง

manga_genres เก็บหนึ่งแถวต่อหนึ่งแนว

API ตรวจชื่อแนวก่อนบันทึก

ภาพปกคนละขั้น

manga_cover_upload.php รับไฟล์แบบ multipart

manga.cover_filename เก็บชื่อไฟล์

manga_cover.php ส่งภาพหลังตรวจการเข้าถึง

ตรวจเจ้าของก่อนเปลี่ยน

manga_update/delete.php ตรวจ manga_id และ user_id

ค่าที Flutter ส่งมาไม่ใช้แทน user_id

หลังสำเร็จหน้ารายละเอียด/รายการปรับข้อมูลใหม่

เส้นทางเมือใช้งานจริง

Dialog → MangaFormResult → add/update.php → manga + manga_genres → cover_upload.php → reload UI

33 | รายละเอียด · คะแนน · รีพอร์ต

หน้ารายละเอียดใช้ข้อมูล Manga ทีรับมา แล้วโหลดตอนและทำงานต่อจากปุ่มต่าง ๆ

เปิดดูภาพหน้าจอเดิม: 11–13

โครงสร้าง Flutter · ตัวแปรและ callback

lib/manga_detail.dart → MangaDetailPage

รับ manga, accessToken, canEdit, canReport

State: _manga, _chapters, _loadingChapters, _busy

build() วางปก ข้อมูล คะแนน และรายการตอน

_loadChapters()

initState() ขอ chapters_get.php?manga_id=...

ตอบกลับ chapters[] แล้ว setState()

เจ้าของเห็นปุ่มเพิ่ม/แก้ไข/ลบตอน

_rate(int rating)

ดาวทีแตะส่ง manga_id + rating

_request('manga_rate.php', ...) ใช้ Bearer

เอา response merge เข้า _manga เพื่ออัปเดตดาว

_report() และ CommentsPage

Dialog รับเหตุผล 10–1000 ตัวอักษร

ส่ง manga_id + reason ไป report_manga.php

ปุ่มความคิดเห็นเปิด CommentsPage(mangaId)

ขอบเขตข้อมูล · API และผลลัพธ์

GET chapters_get.php

คัดตอนของเรืองตามสิทธิและ is_published

response: chapter_id, chapter_number, title ฯลฯ

รายการนีส่งต่อให้ MangaReaderPage

คะแนน

manga_rate.php บันทึกคะแนนราย user ใน manga_ratings

response ส่งค่าเฉลีย/จำนวน/คะแนนของฉัน

manga_get.php ส่งค่ารวมสำหรับการ์ดและรายละเอียด

รายงานผลงาน

report_manga.php บันทึกเหตุผลใน manga_reports

ผู้ใช้ส่งเรืองให้ admin พิจารณา

การส่งรายงานไม่ซ่อนหรือลบเรืองทันที

การแสดงปุ่มกับสิทธิ์จริง

canEdit/canReport คุม UI ทีมองเห็น

ทุก endpoint ตรวจ token และสิทธิของตัวเอง

ห้ามเชือเพียงค่าบูลีนจาก Flutter

เส้นทางเมือใช้งานจริง

MangaCard → MangaDetailPage → chapters_get.php → รายการตอน / rate / report / comments

34 | ตอน · PDF · ชุดภาพ

เรืองหนึงมีหลายตอน และตอนหนึงมีไฟล์ PDF เดียวหรือภาพเรียงหลายหน้า

เปิดดูภาพหน้าจอเดิม: 14

โครงสร้าง Flutter · ตัวแปรและ callback

lib/chapter_dialog.dart

ChapterDialog(nextNumber, chapter?) ใช้เพิม/แก้

State เก็บเลขตอน ชือ published และ files

_pickFiles() เลือก PDF หรือภาพหลายไฟล์

_pickFiles() / _submit()

PDF 1 ไฟล์ ≤20 MB หรือภาพ ≤15 ไฟล์

ภาพแต่ละไฟล์ ≤5 MB; รวมต่อครัง ≤30 MB

คืน ChapterFormResult ผ่าน Navigator.pop

lib/manga_detail.dart → _saveChapter()

สร้าง: chapter_add.php รับเลขตอนและชือ

ถ้ามีไฟล์: uploadChapterFiles(chapterId, ...)

สุดท้าย chapter_update.php ตัง published

lib/chapter_upload.dart

สร้าง http.MultipartRequest

fields: chapter_id, content_type

แนบ pdf หรือ pages[] ตามประเภท

ขอบเขตข้อมูล · API และผลลัพธ์

ตาราง chapters

chapters.manga_id ผูกตอนกับเรือง

chapter_number ใช้เรียง; is_published คุมการเห็น

chapter_id จาก chapter_add.php ส่งกลับ Flutter

POST chapter_upload.php

ตรวจเจ้าของเรืองผ่าน token และ chapter_id

manga_storage.php ตรวจจำนวน ขนาด MIME/ชนิดจริง

บันทึกข้อมูลไฟล์ใน chapter_files ตาม sort_order

ทีเก็บไฟล์

PDF/ภาพอยู่ใน mymangalist_uploads นอก htdocs

chapter_files เก็บ filename, mime_type, byte_size

การอ่านต้องผ่าน manga_file.php

แก้/ลบตอน

chapter_update.php เปลียนเลข ชือ การเผยแพร่

chapter_delete.php ลบตอนและไฟล์ทีเกียวข้อง

ถ้าอัปโหลดพลาด ตอนใหม่อาจคงเป็นร่างไว้

เส้นทางเมือใช้งานจริง

ChapterDialog → chapter_add.php → chapter_id → chapter_upload.php → chapter_update.php → _loadChapters()

35 | หน้าอ่าน · ตั้งค่า · เปลี่ยนตอน

ตัวอ่านรับ chapter_id แล้วขอรายการไฟล์ ก่อนขอไบต์จริงทีละไฟล์ผ่าน endpoint ที่ตรวจสิทธิ์

เปิดดูภาพหน้าจอเดิม: 15–17

โครงสร้าง Flutter · ตัวแปรและ callback

lib/manga_reader.dart → MangaReaderPage

รับ mangaId, chapterId, chapters, chapterIndex

State: _type, _files, _currentPage, _pageCount

initState() เรียก _load()

_load() / _fileUri(fileId)

GET chapter_files_get.php เพื่อรู้ content_type

PDF: GET manga_file.php แล้ว PdfDocument.openData

ภาพ: ใช้ file_id เป็น URL ภาพพร้อม Bearer header

_reader() / _showSettings()

_singlePage เลือกโหมดทีละหน้า/เลื่อนต่อเนื่อง

_fitWidth และ _pageWidth คุมขนาดที่แสดง

_tapToTurn() แบ่งซีกซ้าย/ขวาเพื่อเปลี่ยนหน้า

_openPrevious/NextChapter()

ใช้ chapterIndex หาเพื่อนบ้านใน chapters[]

Navigator.pushReplacement เปิดตอนที่เลือก

_recordView() ส่ง chapter_id หลังโหลดสำเร็จ

ขอบเขตข้อมูล · API และผลลัพธ์

GET chapter_files_get.php

response: content_type และ files[]

แต่ละไฟล์มี file_id, sort_order, mime_type

PHP ตรวจว่า user มองเห็นตอนนี้ได้

GET manga_file.php?file_id=...

PHP หา chapter → manga → สิทธิ์การอ่าน

ส่ง bytes ของ PDF/ภาพจากที่เก็บนอก htdocs

เปลี่ยน URL/file_id ไม่ได้ข้ามการตรวจสิทธิ์

POST manga_view.php

request JSON: chapter_id

server บันทึกยอดชมใน manga_views

หากบันทึกยอดชมพลาด UI ยังอ่านต่อได้

ตัวเลือกการอ่านอยู่ที่ใด

ค่าโหมดและความกว้างอยู่ใน State ของหน้านี้

ไม่ได้แก้ไฟล์ PDF หรือภาพต้นฉบับ

เปลี่ยนตอนแล้วสร้าง MangaReaderPage ใหม่

เส้นทางเมื่อใช้งานจริง

แตะตอน → files_get.php → file_id → manga_file.php → _reader() → แตะซ้าย/ขวา หรือเปลี่ยนตอน

36 | ความคิดเห็น · ระดับเรื่องและตอน

ใช้ Widget เดียวกัน โดย chapterId เป็นตัวบอกว่าคอมเมนต์อยู่ใต้เรื่องหรือใต้ตอน

เปิดดูภาพหน้าจอเดิม: 18

โครงสร้าง Flutter · ตัวแปรและ callback

lib/comments_page.dart → CommentsPage

รับ mangaId และ chapterId?

State: _comments, _text, _loading

initState() เรียก _load()

_load()

GET comments_get.php?manga_id=...

ถ้าเป็นตอน จะเพิ่ม chapter_id ใน query

แปลง response.comments เป็นการ์ดความคิดเห็น

_add() / _delete(commentId)

_add ส่ง body พร้อม manga_id และ chapter_id?

_delete ส่ง comment_id

หลังสำเร็จ _load() เพื่ออัปเดตรายการ

ตำแหน่งที่เปิดหน้านี้

MangaDetailPage เปิดคอมเมนต์ระดับเรื่อง

MangaReaderPage เปิดคอมเมนต์ของตอน

UI ใช้ can_delete ใน response คุมปุ่มลบ

ขอบเขตข้อมูล · API และผลลัพธ์

GET comments_get.php

ตรวจสิทธิ์มองเห็น Manga/ตอนก่อน

response: comments[] พร้อม username, body

ส่ง can_delete ให้แต่ละแถวตามผู้ใช้ปัจจุบัน

POST comment_add.php

ตรวจข้อความ 1–1000 ตัวอักษร

INSERT comments(manga_id, chapter_id, user_id, body)

user_id มาจาก token ไม่ได้มาจากฟอร์ม

POST comment_delete.php

รับ comment_id แล้วตรวจสิทธิ์ลบฝั่ง PHP

ปุ่มที่ซ่อนใน UI ไม่ใช่มาตรการป้องกัน

ผลสำเร็จแล้ว Flutter โหลดรายการใหม่

ตาราง comments

chapter_id ว่าง = ความคิดเห็นต่อเรื่อง

chapter_id มีค่า = ความคิดเห็นต่อตอน

จึงไม่ต้องสร้างหน้าหรือตารางใหม่สองชุด

เส้นทางเมื่อใช้งานจริง

ปุ่มความคิดเห็น → CommentsPage(mangaId, chapterId?) → get/add/delete.php → comments → _load()

37 | Admin · ผู้ใช้ · คิวรีพอร์ต

แผงผู้ดูแลรวมสองงาน แต่ทุกคำสั่งตัดสินสิทธิ์ด้วย role จากฐานข้อมูลฝั่ง PHP

เปิดดูภาพหน้าจอเดิม: 19–21

โครงสร้าง Flutter · ตัวแปรและ callback

lib/admin_dashboard.dart

AdminDashboardPage(user) เก็บ _users, _reports

_usersView() กับ _reportsView() เป็นสองแท็บ

_get()/_post() ใส่ Bearer token ให้คำขอ

จัดการผู้ใช้

_loadUsers() ขอรายการพร้อมสถิติ

_setSuspended() พักหรือคืนสถานะบัญชี

_deleteUser() ยืนยันก่อนลบบัญชี

จัดการรีพอร์ต

_loadReports() ขอคิว pending/ประวัติ

_review(report, action) ส่งคำตัดสิน

_inspectReport() เปิดเรื่อง; _restoreManga() คืนเรื่อง

ตัวเลือก action

dismissed = ปิดรายงาน

hidden / manga_deleted = ซ่อนหรือลบเรื่อง

user_suspended = พักผู้ลงผลงาน

ขอบเขตข้อมูล · API และผลลัพธ์

GET admin_get_users/reports.php

require_admin.php ตรวจ role จาก require_auth.php

users[] มีจำนวน Manga/คอมเมนต์/สถานะ

reports[] มีเหตุผล เรือง ผู้รายงาน และผลตัดสิน

POST admin_suspend/delete_user.php

server ตรวจเป้าหมายและสิทธิ์ admin

พักบัญชีแล้ว token ของบัญชีนั้นใช้ต่อไม่ได้

ลบบัญชีตามความสัมพันธ์ในฐานข้อมูล

POST admin_review_report.php

รับ report_id + action แล้วทำใน transaction

ซ่อน manga, ลบ manga หรือพัก uploader ตาม action

บันทึก reviewed_by และ reviewed_at

POST admin_set_manga_hidden.php

ใช้คืนเรื่องทีเคยซ่อน

PHP ตรวจ admin ก่อนแก้ is_hidden

UI โหลดคิวและข้อมูลใหม่หลังทำรายการ

เส้นทางเมือใช้งานจริง

Admin tab → _get/_post → require_admin.php → users / manga_reports / manga → reload แท็บ

38 | PHP · สิทธิ · ตารางข้อมูล

ชั้นหลังบ้านเป็นตัวตัดสินข้อมูลที่อ่านหรือแก้ได้ ไม่อาศัยแค่การซ่อนปุ่มใน Flutter

เปิดดูภาพหน้าจอเดิม: 23–25

โครงสร้าง Flutter · ตัวแปรและ callback

รูปแบบ endpoint ใน PHP

ตรวจ HTTP method → require_auth.php

อ่าน JSON หรือ multipart → validate ข้อมูล

prepared statement → JSON success/message/data

Bearer ที่ Flutter ส่ง

header: Authorization: Bearer <access_token>

login.dart ใส่ token ใน user map

ทุกหน้าใช้ widget.user หรือ widget.accessToken

รหัสเชื่อมสามระดับ

manga_id ระบุเรื่อง; chapter_id ระบุตอน

file_id ระบุ PDF หรือภาพหนึ่งไฟล์

Flutter ส่ง id → PHP หาเจ้าของ/สถานะต่อ

ดูโค้ดจากภาพอย่างไร

หาชื่อ Widget ใน lib/ → อ่าน State/ปุ่ม

ตาม callback ไป URL ชื่อ .php

ดู require_auth/admin และ SQL ของ endpoint นั้น

ขอบเขตข้อมูล · API และผลลัพธ์

require_auth.php

ตรวจรูปแบบ token และ SHA-256 hash

JOIN api_tokens กับ users และตรวจ expires_at

ได้ authenticatedUser.user_id, role, is_suspended

require_admin.php / manga_access.php

admin ต้องมี role=admin จากฐานข้อมูล

owner อ่านเรื่องตัวเอง; คนอื่นอ่าน public ที่ไม่ซ่อน

admin เปิดเรื่องที่ถูกรีพอร์ตเพื่อตรวจได้

ตารางหลักใน database.sql

users, api_tokens, manga, manga_genres

chapters, chapter_files, manga_views, manga_ratings

comments, manga_reports

ไฟล์จริง

manga_storage.php เก็บ upload นอก htdocs

manga_file.php ส่งเนื้อหาหลังตรวจสิทธิ์

DB เก็บชื่อไฟล์และ metadata ไม่ใช่ PDF ทั้งก้อน

เส้นทางเมื่อใช้งานจริง

Flutter callback → Bearer + JSON/ไฟล์ → PHP guard → SQL/ไฟล์ → JSON/bytes → setState()